

Normalization Confluence in Federated Registry Networks

Dayna Blackwell

dayna@blackwell-systems.com

Licensed under CC-BY-4.0.

4 October 2026 (Version 2)

Version 2. See Appendix (Errata and corrections) for changes from version 1.

Abstract

Normalization confluence [1] guarantees coordination-free convergence within a single registry: under well-founded compensation (WFC) and compensation commutativity (CC), all processors consuming the same events reach the same valid state regardless of application order. We extend this guarantee to *federated* environments where multiple registries, each with independent invariants and compensation, are connected by *registry morphisms* encoding cross-organizational semantic constraints.

Our key construction treats a federation of registries connected by morphisms as a registry, with morphism consistency conditions as additional invariants and morphism repair as additional compensation. Repair confluence then composes freely: on any acyclic network whose morphisms preserve validity (M1), with a source-determined (R1), validity-preserving (R2) *resolution operator* at each multi-source target, one round of federated normalization reaches the unique federated normal form, from any state. The core mechanism is an *authority argument*: the sources' states determine every target's shared component. Event confluence does not compose for free. A target event can read a shared value that morphism repair overwrites, so component convergence and M1 do not imply federated convergence (we give counterexamples satisfying every component condition). We identify two local, per-edge checks, cross-registry compensation commutativity (C1) and repaired compensation commutativity (C2), and prove that they suffice for convergence of every event interleaving when events are applied to normalized states, and that the same two equations, evaluated at the witnesses a run can reach, are exactly necessary and sufficient.

We prove the structural conditions necessary by counterexample: without acyclicity, contradictory cyclic morphisms cause compensation to cycle; without validity preservation, repair and local compensation oscillate; without source determinacy, the shared normal form depends on the interleaving. On ordered shared domains with monotone repair, cycles are admitted: the repair normal form is the least fixed point (federally valid when bottom is valid), and event interleavings converge exactly when independent events commute after re-normalization, which per-target C1 and C2 over the repair images imply. The federated results are mechanized in Coq/Rocq, axiom-free. For self-containment, we restate the single-registry model and convergence results of [1] alongside a verification calculus for CC.

Keywords: Stream Processing, Normalization Confluence, Semantic Consistency, Rewrite Systems, Newman's Lemma, Compensation Commutativity, Federated Systems, Registry Morphisms, Distributed Systems

1 Introduction

Coordination-free convergence—the ability of distributed processors to agree on a result without explicit synchronization—is among the central goals of distributed systems design. The literature has identified two structural regimes under which it is achievable:

1. **Operation commutativity** (CRDTs [3]): Operations are designed to commute algebraically. Any application order produces the same state.
2. **Invariant confluence** (I-confluence [6]): Operations individually preserve application invariants under all orderings. No repair is needed because validity is never violated.

Both regimes achieve convergence by *preventing* inconsistency. CRDTs prevent it through algebraic design; I-confluence prevents it through invariant-preserving operation selection. Neither accommodates operations that are non-commutative *and* may violate invariants—yet such operations are pervasive in practice. An order processing pipeline, for example, must handle events (orders, payments, inventory adjustments, shipments) that do not commute and that individually violate business rules (no shipment without payment, no negative inventory).

We formalize the missing piece as a *registry*: any authoritative definition of validity external to the stream processor. Registries already exist in practice under various names—policy engines, constraint services, schema validators, compliance controllers—but their role in convergence has not been characterized formally. Our model abstracts these as a tuple of invariant predicates and a compensation operator.

This paper presents a unified treatment of a third regime, **normalization confluence** [1], and extends it to multi-registry environments. In normalization confluence, operations may be non-commutative and may violate invariants. But a compensation operator normalizes states to validity after each operation, and the key structural condition is that these *compensated results* commute—even though the operations themselves do not. Under this condition, together with well-founded compensation, Newman’s Lemma guarantees that all causality-respecting processing orders reach the same valid state.

The three regimes compare as follows:

1. Operation commutativity (every state valid) $\implies$ compensation commutativity $\implies$ convergence.
2. Invariant confluence $\implies$ no compensation needed; in the sequential-application model of this paper, convergence then still requires the operations to commute on the reachable states.
3. Well-founded compensation and compensation commutativity $\implies$ convergence [1].

Normalization confluence occupies the design space between CRDT commutativity (which requires operations to commute) and I-confluence (which requires operations to preserve invariants). It permits operations that satisfy neither, provided their compensated results agree.

Modern distributed systems, however, span organizational boundaries: a supply chain links manufacturers and suppliers, each with its own registry. We show that normalization confluence extends naturally to these *federated* settings. Multiple registries connected by *morphisms*—structured mappings encoding cross-organizational constraints—form a single federated registry. Termination and the repair normal form need nothing beyond the component conditions and validity preservation of the morphisms (on acyclic networks, with *resolution operators*, source-determined and validity-preserving merges, at multi-source targets). Convergence of event interleavings needs more: a target event may read a shared value that a morphism overwrites, and then component convergence does not carry over. Two local checks per edge, C1 and C2, close the gap, and their reachable forms are exactly necessary and sufficient.

Relationship to prior work. This paper extends [1], which establishes the single-registry governance rewrite system, the core convergence result ($\text{WFC} + \text{CC} \Rightarrow \text{unique normal forms}$), necessity and complexity results, and a verification calculus for CC based on invariant footprints and decomposable repair. The present paper adds the federated extension (Section 8): registry morphisms with partial authority, the federated construction, the authority argument, the federated convergence theorem under the checks C1 and C2 with its exact converse and necessity counterexamples, and its generalization to multi-source acyclic networks via resolution operators (Section 8.5). For self-containment, we restate the single-registry model and include all proofs needed to support the federated result; readers familiar with [1] may proceed directly to Section 8.

1.1 Convergence via Normalization

Convergence is conventionally treated as a property of *operations*: operations must be designed to commute, or designed to preserve invariants. Our result reframes convergence as a property of the *normalization rewrite system*: the system that interleaves event application with invariant repair. The operations themselves need no special algebraic structure. What matters is that the rewrite system—encompassing both application and compensation—is terminating and confluent. This shifts the design burden from operation algebra to compensation design.

1.2 Contributions

The single-registry results (items 1–6) are from [1], restated here for self-containment. The federated extension (items 7–10) is new.

1. **Normalization confluence regime** (Section 3): A governance rewrite system on configurations (σ, B) pairing a state with a pending-event buffer, with rules for both event application and compensation. Nondeterminism arises from the choice of which enabled event to apply next.
2. **Convergence theorem** (Section 5): Proof that processors consuming the same events converge to the same valid state, regardless of application order, under well-founded compensation (WFC) and compensation commutativity (CC).
3. **Tightness and exactness** (Section 6): without UBC, compensation depth is unbounded; a failure of CC can make different orders reach different valid states; and CC is the exact condition (an if-and-only-if) when every buffered event may fire and compensation repairs to validity, with a joinability form of CC exact under any enabledness.
4. **Complexity analysis** (Section 7): Proof that registry governance adds $O(\log n)$ overhead per event under UBC, preserving practical scalability.
5. **Three-regime characterization** (Section 9): Together with CRDTs and invariant confluence, a partition of the coordination-free design space into operation commutativity, invariant confluence, and normalization confluence.
6. **Verification calculus** (Section 10): invariant footprints and decomposable repair reduce the global CC obligation to per-event checks (strong absorption, footprint absorption) and per-pair commutation up to normalization, with each hypothesis shown to be needed. Product composition enables modular verification of independent registries.
7. **Federated convergence** (Section 8): Extension to tree-shaped multi-registry networks connected by morphisms. Registry morphisms with partial authority model cross-organizational constraints. Federated termination and a unique repair normal form are derived from network structure via an authority argument, with only morphism validity preservation (M1)

needed. Event convergence is not derived: component CC and M1 do not suffice (counterexamples). Under two per-edge checks, C1 and C2, every event interleaving applied to normalized states converges (Theorem 8.12); the same equations at reachable witnesses are the exact condition. Necessity of acyclicity, M1, C1 and C2 is shown by counterexample.

8. **Multi-source resolution** (Section 8.5): Removal of the tree restriction. A target with several incoming morphisms carries a *resolution operator* generalizing the authority function; under source determinacy and validity preservation (R1–R2), the repair normal form is unique on any acyclic network, and event convergence holds under C1 and C2 (Theorem 8.24), with the tree theorem as the single-source special case. R1, R2, C1 and C2 are finite conditions, checkable by enumeration, so a resolved network is certified as WFC and CC are.
9. **Monotone cycles** (Section 8.6): When shared domains are ordered lattices and repair is monotone, acyclicity is unnecessary for the repair normal form: it is the least fixed point (Knaster–Tarski), reached by every fair chaotic iteration under the ascending chain condition, and federally valid when every component is valid at bottom (Theorem 8.29). Event interleavings on such a network converge exactly when independent events commute after re-normalization (the global condition GC), which per-target C1 and C2 over the repair images imply. Acyclicity and monotonicity are two independent routes to a unique repair normal form.
10. **Compositionality** (Section 8.7): A convex, internally convergent sub-federation collapses to a single *effective registry* (Theorem 8.37), with internal convergence now including C1 and C2; the outer network and the collapsed one have the same repair normal form and the same runs, so one converges iff the other does. This gives modular and hierarchical verification, generalizing product composition from independent registries to sub-diagrams.
11. **Machine-checked results** (Remark 5.4, Remark 8.40): The single-registry Convergence Theorem and the federated results above, including every counterexample and the exact converses, are mechanized in Coq/Rocq, axiom-free (`Print Assumptions` reports “Closed under the global context” for every theorem in the gate), with concrete instances witnessing that the hypotheses are jointly satisfiable.

2 Related Work

Conflict-Free Replicated Data Types. CRDTs [3] guarantee convergence through algebraic structure: replicas that apply the same set of commutative, associative, idempotent operations converge to the same state. Our setting is strictly more general in one dimension: we do not require operations to commute, only that their *compensated results* commute. This permits operations that individually violate invariants, which CRDTs cannot accommodate. Conversely, CRDTs require no finiteness or boundedness assumptions. Neither approach dominates: CRDTs are preferable when operations can be designed to commute, while registry governance is preferable when business rules constrain valid states in ways that preclude commutativity.

CALM and Coordination Avoidance. The CALM theorem [4, 5] establishes that monotone programs can achieve consistency without coordination. Bailis et al. [6] extend this to identify *invariant confluence*: a set of operations is invariant-confluent if all possible orderings preserve application invariants. When invariant confluence holds, no compensation is needed. Our framework addresses the complement: operations that are *not* invariant-confluent. We prove that compensation restores convergence under well-founded compensation and compensation commutativity. Section 9.1 compares the two regimes precisely.

Invariant-Repair Systems. Indigo [16] and IPA [17] also restore application invariants over weakly consistent replicas, and Indigo in particular targets the cross-object constraints that arise across service boundaries. They do so by a different route: static analysis of invariant-violating operation pairs, to which they attach application-specific conflict-resolution rules or reservation (escrow) protocols so that every reachable state stays invariant-preserving, in the invariant-confluence tradition. We do not preselect operations to be invariant-preserving. We admit operations that genuinely violate invariants and prove, via Newman’s Lemma under compensation commutativity, that repair reconciles all orderings to the same valid normal form; the federated extension carries this across registry morphisms. The distinction is one of method: reservation and invariant confluence versus confluence of a compensating rewrite system.

Abstract Rewrite Systems. Newman’s Lemma [2] states that a terminating abstract rewrite system is confluent if and only if it is locally confluent. The lemma has been applied extensively in term rewriting [7] and lambda calculus. Our contribution is its application to a rewrite system with *genuine nondeterminism*: the choice of which event to apply next from a buffer of enabled events. Local confluence is nontrivial because different application orders produce different intermediate states; our compensation commutativity axiom is the condition that closes the resulting critical pairs.

Rewriting Logic and Coherence. Meseguer’s rewriting logic [18] models concurrent systems as rewrite theories, and coherence checking (the Church-Rosser and coherence properties of rewrite theories modulo equations [19], supported by tools such as Maude) is the closest methodological ancestor of our build-time convergence check. We instantiate this lineage for one specific rewrite system: events interleaved with invariant compensation, where compensation commutativity plays the role of the coherence condition that closes critical pairs.

Self-Stabilization. Self-stabilizing systems [20] converge to a legitimate state from an arbitrary starting configuration. Normalization confluence shares the converge-to-valid premise but strengthens the conclusion to agreement: not merely that each replica reaches some valid state, but that all replicas, across a federation, reach the *same* valid state under reordering.

Eventual Consistency. Classical eventual consistency [8] guarantees that replicas converge given sufficient time without new updates. It does not address whether converged states satisfy application-level invariants—only that replicas agree on the same value. Our work guarantees convergence to *valid* states under continuous event arrival.

Stream Processing Theory. Formal models of stream processing include synchronous dataflow [10], Kahn process networks [11], and the windowed computation models underlying modern engines [12]. These focus on operational semantics—what a stream processor computes—rather than semantic validity of computed states. Our registry layer sits above any of these operational models.

Saga Pattern and Compensating Transactions. The saga pattern [9] decomposes long-lived transactions into sequences of local transactions with compensating actions for rollback. Our compensation operator generalizes this: rather than undoing individual operations, it repairs arbitrary invariant violations in the current state. The convergence guarantee we prove—that different interleaving orders of application and compensation reach the same valid state—has no analogue in the saga literature.

Mechanized Convergence Proofs. Gomes et al. [21] give a machine-checked proof of strong eventual consistency for CRDTs in Isabelle/HOL, network model included. Their result covers the commutative case; the mechanization this work builds on (Remark 5.4) covers convergence under non-commutative, invariant-violating operations reconciled by compensation. Both rest on an assumption-free, machine-checked convergence theorem rather than a paper proof.

3 Registry-Governed Stream Model

3.1 Registries and Semantic Projection

Definition 3.1 (Registry). A *registry* is a tuple $R = (\Sigma_R, I_R, V_R, \rho_R)$ where:

- Σ_R is a set of *semantic states*,
- $I_R = \{\psi_1, \dots, \psi_k\}$ is a finite set of *invariant predicates* $\psi_i : \Sigma_R \rightarrow \{\top, \perp\}$,
- $V_R : \Sigma_R \rightarrow \{\top, \perp\}$ is the *validation function* defined by $V_R(\sigma) = \bigwedge_{i=1}^k \psi_i(\sigma)$,
- $\rho_R : \Sigma_R \rightarrow \Sigma_R$ is a *deterministic compensation operator* such that if $V_R(\sigma) = \top$, then $\rho_R(\sigma) = \sigma$.

Definition 3.2 (Semantic Projection). A registry is defined over a semantic state space Σ_R together with a *projection* $\pi : \Sigma_{\text{raw}} \rightarrow \Sigma_R$ from unbounded raw states to semantic states. All invariants, validation, and compensation are defined on Σ_R . Two raw states s_1, s_2 are *registry-indistinguishable* if $\pi(s_1) = \pi(s_2)$. The application function $\text{apply} : \mathcal{E} \times \Sigma_R \rightarrow \Sigma_R$ is understood as “apply then project”: given a raw-level application $\text{apply}_{\text{raw}} : \mathcal{E} \times \Sigma_{\text{raw}} \rightarrow \Sigma_{\text{raw}}$, the semantic application is $\text{apply}(e, \sigma) = \pi(\text{apply}_{\text{raw}}(e, s))$ for any s with $\pi(s) = \sigma$. Well-definedness requires that the projected result is independent of the representative s .

Remark 3.3 (Scope of Finiteness). Finiteness is required of the *semantic* state space Σ_R , not the raw data. An order-processing registry tracking status across states *pending*, *paid*, *shipped*, *delivered*, *cancelled* has $|\Sigma_R| = 5$ regardless of how many bytes the underlying order record occupies. More generally, any domain whose logic is captured by k boolean predicates has $|\Sigma_R| \leq 2^k$: the semantic states are the equivalence classes under the invariant vector $(\psi_1(\sigma), \dots, \psi_k(\sigma))$. Business domains are overwhelmingly of this form. The finiteness assumption applies to these semantic equivalence classes, not to raw data representations.

Definition 3.4 (Registry Equivalence). Two states $\sigma_1, \sigma_2 \in \Sigma_R$ are *registry equivalent*, written $\sigma_1 \approx_R \sigma_2$, if they satisfy the same invariants: $\forall \psi \in I_R : \psi(\sigma_1) = \psi(\sigma_2)$.

Lemma 3.5. *Registry equivalence $\approx_R$ is an equivalence relation on Σ_R .*

Proof. Each condition $\psi(\sigma_1) = \psi(\sigma_2)$ is reflexive, symmetric, and transitive. A conjunction of equivalence relations is an equivalence relation. $\square$

3.2 Well-Founded and Uniformly Bounded Compensation

Termination and complexity require two properties of the compensation operator, which we state separately because they play different roles.

Axiom 3.6 (Well-Founded Compensation (WFC)). There exists a *compensation measure* $\Phi : \Sigma_R \rightarrow \mathbb{N}$, defined over semantic states (after projection), such that every compensation step strictly decreases Φ : if $V_R(\sigma) = \perp$, then $\Phi(\rho_R(\sigma)) < \Phi(\sigma)$.

WFC guarantees that compensation terminates from any state: iterated application of ρ_R reaches a valid state in finitely many steps.

Axiom 3.7 (Uniformly Bounded Compensation (UBC)). The compensation measure Φ from Axiom 3.6 is uniformly bounded: there exists $M \in \mathbb{N}$ such that $\Phi(\sigma) \leq M$ for all $\sigma \in \Sigma_R$.

UBC strengthens WFC by bounding compensation depth *uniformly* across all states. UBC does not require finiteness of Σ_R ; bounded compensation height alone suffices. WFC alone suffices for termination and for defining iterated compensation ρ_R^* . UBC is additionally needed for the uniform step bound in the termination lemma and the constant per-event compensation cost in the complexity analysis.

Lemma 3.8 (Finite State Spaces Imply UBC). *If $|\Sigma_R| < \infty$ and WFC holds, then UBC holds with $M = \max_{\sigma \in \Sigma_R} \Phi(\sigma)$.*

Proof. The maximum exists because Σ_R is finite. □

Remark 3.9. WFC is the load-bearing condition for termination; UBC is the load-bearing condition for uniform step bounds and per-event complexity. Finiteness of Σ_R is a convenient sufficient condition for both, but UBC does not require full finiteness of Σ_R ; it requires only bounded compensation height. The counterexample in Section 6 satisfies WFC (compensation terminates for every fixed event set) but violates UBC (compensation depth grows without bound as event sets grow).

3.3 Events and Streams

Definition 3.10 (Event Stream). An *event stream* $\mathcal{S} = \langle e_1, e_2, \dots \rangle$ is a (possibly infinite) sequence of events drawn from event space $\mathcal{E}$. Each event e_i carries a unique identifier $\text{id}(e_i)$, a timestamp $\tau(e_i)$, and a set of causal dependencies $\text{deps}(e_i) \subseteq \{e_1, \dots, e_{i-1}\}$. An *application function* $\text{apply} : \mathcal{E} \times \Sigma_R \rightarrow \Sigma_R$ maps each event and current state to a new state.

Definition 3.11 (Causality-Respecting Order). An ordering π of a finite event set E *respects causality* if whenever $e_i \in \text{deps}(e_j)$, event e_i precedes e_j under π . Multiple causality-respecting orders may exist for the same event set.

3.4 Governance Rewrite System

We now define the rewrite system that is the core technical device of this paper. Unlike the approach of fixing a canonical event order—which would make agreement trivial by determinism—our rewrite system admits genuine nondeterminism in the choice of which event to apply next.

Assumption 3.12 (Causal Closure). The received event set E is causally closed: for every $e \in E$, $\text{deps}(e) \subseteq E$.

Under causal closure, every dependency of every event in E is either still pending in B or has already been applied (removed from B). This makes the enabled predicate below well-defined.

Definition 3.13 (Configuration). A *configuration* is a pair (σ, B) where $\sigma \in \Sigma_R$ is a semantic state and $B \subseteq E$ is a set of events that have been received but not yet applied. An event $e \in B$ is *enabled* if $\text{deps}(e) \cap B = \emptyset$ (all causal dependencies of e have already been applied).

Definition 3.14 (Governance Rewrite System). Fix a finite set of received events E and initial state σ_0 . The *governance rewrite system* $\mathcal{G} = (\mathcal{C}, \rightarrow)$ operates on configurations (σ, B) with $\sigma \in \Sigma_R$ and $B \subseteq E$. The rewrite relation $\rightarrow$ is the union of:

1. **Apply step:** $(\sigma, B) \rightarrow (\text{apply}(e, \sigma), B \setminus \{e\})$ whenever $e \in B$ is enabled.
2. **Compensation step:** $(\sigma, B) \rightarrow (\rho_R(\sigma), B)$ whenever $V_R(\sigma) = \perp$.

A configuration is in *normal form* if $B = \emptyset$ and $V_R(\sigma) = \top$.

The nondeterminism is genuine: when multiple events are enabled and the state is invalid, a configuration may step via different apply rules (choosing different events) or via the compensation rule. Confluence of this system means that the choice does not affect the final result.

3.5 Compensation Commutativity

Local confluence of $\mathcal{G}$ is nontrivial. Two critical pair types arise: (1) two apply steps choosing different events, and (2) an apply step versus a compensation step. The following axiom ensures both types close.

Definition 3.15 (Iterated Compensation). Under WFC (Axiom 3.6), define $\rho_R^*(\sigma)$ to be the unique state obtained by iterating ρ_R from σ until reaching validity: $\rho_R^*(\sigma) = \rho_R^m(\sigma)$ for the least m such that $V_R(\rho_R^m(\sigma)) = \top$. Existence and finiteness of m follow from WFC (indeed $m \leq \Phi(\sigma)$); uniqueness follows from determinism of ρ_R . Because V_R is Boolean-valued, ρ_R^* is computable, and it does not depend on which measure witnesses WFC.

Axiom 3.16 (Compensation Commutativity (CC)). The registry R satisfies:

(CC1) Order independence. For any $\sigma \in \Sigma_R$ and events e_1, e_2 that are causally independent and simultaneously enabled in some configuration (σ, B) :

$$\rho_R^*(\text{apply}(e_2, \rho_R^*(\text{apply}(e_1, \sigma)))) = \rho_R^*(\text{apply}(e_1, \rho_R^*(\text{apply}(e_2, \sigma)))).$$

(CC2) Compensation absorption. For any $\sigma \in \Sigma_R$ with $V_R(\sigma) = \perp$ and any enabled event e :

$$\rho_R^*(\text{apply}(e, \sigma)) = \rho_R^*(\text{apply}(e, \rho_R(\sigma))).$$

Remark 3.17 (CRDT Commutativity). CRDT commutativity requires $\text{apply}(e_1, \text{apply}(e_2, \sigma)) = \text{apply}(e_2, \text{apply}(e_1, \sigma))$ for all σ . This implies CC1 (take $\rho_R^* = \text{id}$ when all states are valid) and trivially satisfies CC2 (when all states are valid, compensation never fires). Compensation commutativity is strictly weaker: it permits operations that individually violate invariants, requiring only that the *repaired* results commute. The gap between operation commutativity and compensation commutativity is precisely the space our result occupies.

Remark 3.18 (Compensation Absorption). Condition CC2 states that compensating an invalid state before applying an event does not change the final compensated result. Intuitively, the repair that compensation performs “would have been performed anyway” during the post-application compensation phase. CC2 holds whenever compensation repairs invariant violations in a way that is determined by the violation pattern rather than the path taken to reach the violation. By induction on the number of compensation steps, CC2 generalizes: $\rho_R^*(\text{apply}(e, \sigma)) = \rho_R^*(\text{apply}(e, \rho_R^*(\sigma)))$. In rewrite-theoretic terms, CC2 ensures that the normalizer is stable under interleaving with event application: normalizing “early” (before an event) versus “late” (after) does not change the final normal form.

3.6 Stream Processors

Definition 3.19 (Stream Processor). A *stream processor* P for stream $\mathcal{S}$ under registry R is a process that, at each time t , has received some subset of events $E_P(t) \subseteq \mathcal{S}$ and maintains state $\Psi_P(t) \in \Sigma_R$. We require:

1. **Eventual delivery:** For every event $e_i \in \mathcal{S}$, there exists a time t_i such that $e_i \in E_P(t)$ for all $t \geq t_i$.
2. **Causality-respecting application:** The processor applies events in some order that respects causality (Definition 3.11), buffering events whose causal dependencies have not yet been received. The processor is free to choose any causality-respecting order.
3. **Incremental compensation:** After applying each event, the processor applies ρ_R until validity is restored.

The processor’s computation on event set $E_P(t)$ is a reduction sequence in the governance rewrite system $\mathcal{G}$ starting from $(\sigma_0, E_P(t))$. The processor has *finished processing* $E_P(t)$ at time t if its configuration at t is a normal form of that reduction. It is *fair* if, while no new events arrive, it keeps reducing, and makes progress whenever it has not finished.

A received set that is not yet causally closed (an event has arrived before one of its dependencies) is never finished in this sense: the waiting event stays in the buffer, so the configuration is not a normal form. Different processors may apply the same events in different causality-respecting orders. The convergence theorem guarantees that, once they have finished processing the same events, they hold the same valid state.

4 Worked Example: Order Fulfillment

We instantiate the framework on an order fulfillment pipeline, verify the structural conditions, and trace two processor orderings to the same valid state.

4.1 Registry Definition

An order has a *stage* in $\{\text{pending}, \text{approved}, \text{held}\}$ and a *balance* in $\{0, 1, 2, 3\}$. The semantic state space is $\Sigma_R = \{\text{pending}, \text{approved}, \text{held}\} \times \{0, 1, 2, 3\}$, with $|\Sigma_R| = 12$ and initial state $\sigma_0 = (\text{pending}, 0)$.

A single invariant governs fulfillment:

$$\psi(s, b) = \top \iff (s = \text{approved} \implies b \geq 2).$$

Approval requires a balance of at least 2. The only invalid states are $(\text{approved}, 0)$ and $(\text{approved}, 1)$.

Compensation. When an order is approved with insufficient balance, it is placed on hold:

$$\rho(\text{approved}, b) = (\text{held}, b) \quad \text{for } b < 2.$$

All other states are valid, and ρ acts as the identity. The *held* stage preserves the intent to approve: the system records that approval was requested but cannot yet proceed. This is a deliberate design choice—a naive alternative that simply cancels the approval (reverting to *pending*) would violate CC, as we discuss below.

Measures. $\Phi(s, b) = 1$ if $s = \text{approved}$ and $b < 2$, otherwise 0. WFC holds (ρ strictly decreases Φ on invalid states), and UBC holds with $M = 1$.

4.2 Events

Two causally independent events:

- e_{credit} : Adds 1 to the balance (capped at 3). If the order is on hold and the new balance reaches 2, the order is automatically approved:

$$\text{apply}(e_{\text{credit}}, (s, b)) = \begin{cases} (\text{approved}, \min(b+1, 3)) & \text{if } s = \text{held} \text{ and } \min(b+1, 3) \geq 2, \\ (s, \min(b+1, 3)) & \text{otherwise.} \end{cases}$$

- e_{approve} : Sets the stage to *approved*: $\text{apply}(e_{\text{approve}}, (s, b)) = (\text{approved}, b)$.

4.3 Verifying CC

CC1 (Order Independence). We verify CC1 for $(e_{\text{approve}}, e_{\text{credit}})$ at two representative starting states.

From $(\text{pending}, 0)$:

Path 1 (e_{approve} first):

$$(\text{pending}, 0) \xrightarrow{e_{\text{approve}}} (\text{approved}, 0) \xrightarrow{\rho^*} (\text{held}, 0) \xrightarrow{e_{\text{credit}}} (\text{held}, 1). \quad \text{Result: } (\text{held}, 1).$$

Path 2 (e_{credit} first):

$$(\text{pending}, 0) \xrightarrow{e_{\text{credit}}} (\text{pending}, 1) \xrightarrow{e_{\text{approve}}} (\text{approved}, 1) \xrightarrow{\rho^*} (\text{held}, 1). \quad \text{Result: } (\text{held}, 1). \checkmark$$

In Path 1, approval fires first, violates the invariant (balance $0 < 2$), and compensation places the order on hold. The subsequent credit increments the balance but does not yet reach the threshold. In Path 2, credit arrives first (no violation), then approval violates the invariant and compensation again produces the held state. Both paths reach $(\text{held}, 1)$.

From $(\text{pending}, 1)$:

$$\text{Path 1: } (\text{pending}, 1) \xrightarrow{e_{\text{approve}}} (\text{approved}, 1) \xrightarrow{\rho^*} (\text{held}, 1) \xrightarrow{e_{\text{credit}}} (\text{approved}, 2). \quad \text{Result: } (\text{approved}, 2).$$

$$\text{Path 2: } (\text{pending}, 1) \xrightarrow{e_{\text{credit}}} (\text{pending}, 2) \xrightarrow{e_{\text{approve}}} (\text{approved}, 2). \quad \text{Result: } (\text{approved}, 2). \checkmark$$

In Path 1, approval again violates the invariant; compensation holds; the subsequent credit reaches balance 2, triggering auto-approval. In Path 2, the credit arrives first, and approval at balance 2 is valid. Both reach $(\text{approved}, 2)$: the order is approved with sufficient funds.

Since $|\Sigma_R| = 12$, CC1 can be verified exhaustively; it holds for every pair of events at all 12 states, invalid ones included. All cases follow the same pattern: compensation preserves the approval intent in the held state, and subsequent credits eventually satisfy the balance threshold.

CC2 (Compensation Absorption). From the invalid state $(approved, 0)$ with event e_{credit} :

LHS: $\rho^*(\text{apply}(e_{\text{credit}}, (approved, 0))) = \rho^*((approved, 1)) = (held, 1)$.

RHS: $\rho^*(\text{apply}(e_{\text{credit}}, \rho(approved, 0))) = \rho^*(\text{apply}(e_{\text{credit}}, (held, 0))) = \rho^*((held, 1)) = (held, 1)$. ✓

Compensating before the credit (putting the order on hold first) produces the same result as applying the credit to the invalid state and then compensating. CC2 holds because the held state and the approved-but-invalid state respond identically to subsequent credits after compensation.

4.4 Two Processors, Three Events

Consider the stream $\mathcal{S} = \langle e_{\text{approve}}, e_{\text{credit}}, e_{\text{credit}} \rangle$ (approval followed by two credits). Two processors P_1, P_2 receive the events in different orders:

Processor P_1 (approve, credit, credit):

$$(pending, 0) \xrightarrow{e_{\text{approve}}} (approved, 0) \xrightarrow{\rho^*} (held, 0) \xrightarrow{e_{\text{credit}}} (held, 1) \xrightarrow{e_{\text{credit}}} (approved, 2).$$

Processor P_2 (credit, credit, approve):

$$(pending, 0) \xrightarrow{e_{\text{credit}}} (pending, 1) \xrightarrow{e_{\text{credit}}} (pending, 2) \xrightarrow{e_{\text{approve}}} (approved, 2).$$

Both processors arrive at $(approved, 2)$. P_1 encountered an invariant violation (approval without sufficient balance) and passed through the held state; P_2 never violated the invariant. Compensation commutativity guarantees agreement regardless of the path.

4.5 Why Compensation Design Matters

The held stage is essential. A naive compensation that simply reverts approval to *pending*, that is, $\rho(approved, b) = (pending, b)$ for $b < 2$, satisfies WFC and UBC but violates CC1. The witness is the state $(pending, 1)$: approval then credit gives $\rho(approved, 1) = (pending, 1)$ and then $(pending, 2)$ (intent to approve lost), while credit then approval gives $(pending, 2)$ and then $(approved, 2)$, which is valid. So

$$\begin{aligned} \rho^*(\text{apply}(e_{\text{credit}}, \rho^*(\text{apply}(e_{\text{approve}}, (pending, 1))))) &= (pending, 2), \\ \rho^*(\text{apply}(e_{\text{approve}}, \rho^*(\text{apply}(e_{\text{credit}}, (pending, 1))))) &= (approved, 2) \neq (pending, 2). \end{aligned}$$

On the stream $\langle e_{\text{approve}}, e_{\text{credit}}, e_{\text{credit}} \rangle$ below, P_1 would end in $(pending, 2)$ and P_2 in $(approved, 2)$. (At $(pending, 0)$ the two orders happen to agree: both give $(pending, 1)$.) The naive revert also violates CC2: at the invalid state $(approved, 1)$,

$$\begin{aligned} \rho^*(\text{apply}(e_{\text{credit}}, (approved, 1))) &= (approved, 2), \\ \rho^*(\text{apply}(e_{\text{credit}}, \rho(approved, 1))) &= (pending, 2). \end{aligned}$$

The held stage preserves the approval intent so that subsequent credits can trigger auto-approval, ensuring that different orderings converge.

This illustrates the practical content of the CC conditions: they constrain *compensation design*, not operation design. What the held stage provides is *absorption*: compensating before a credit or after it gives the same normalized result (CC2), and the two events commute once results are normalized. Section 10 shows that these per-event and per-pair facts give CC1 at every state, and that under absorption CC1 is equivalent to commutation up to normalization (Theorem 10.10).

5 Convergence Theorem

5.1 Termination

Lemma 5.1 (Termination). *Under WFC (Axiom 3.6), the governance rewrite system $\mathcal{G}$ is terminating: every reduction sequence from (σ_0, E) is finite. Under UBC (Axiom 3.7), the total number of steps is bounded by $|E| + (|E| + 1) \cdot M$.*

Proof. Define the measure $\mu(\sigma, B) = (|B|, \Phi(\sigma))$ taking values in $\mathbb{N} \times \mathbb{N}$ under the lexicographic order, which is well-founded.

An apply step sends (σ, B) to $(\text{apply}(e, \sigma), B \setminus \{e\})$. The first component decreases: $|B \setminus \{e\}| = |B| - 1$. Therefore μ strictly decreases regardless of the second component.

A compensation step sends (σ, B) to $(\rho_R(\sigma), B)$. The first component is unchanged; the second strictly decreases by WFC. Therefore μ strictly decreases.

Since μ maps into a well-ordered set and strictly decreases at every step, every reduction sequence is finite. Under UBC, $\Phi(\sigma) \leq M$ for all σ , so at most M compensation steps can occur between apply steps (including before the first apply), giving the uniform bound $|E| + (|E| + 1) \cdot M$. $\square$

The bound is tight: a registry with $M = 1$ and a single event has a reduction of $3 = |E| + (|E| + 1)M$ steps (compensate, apply, compensate).

5.2 Local Confluence

Lemma 5.2 (Local Confluence). *Under CC (Axiom 3.16), the governance rewrite system $\mathcal{G}$ is locally confluent: if $(\sigma, B) \rightarrow X_1$ and $(\sigma, B) \rightarrow X_2$, then there exists a configuration Z with $X_1 \rightarrow^* Z$ and $X_2 \rightarrow^* Z$.*

Proof. Three cases arise from the two rewrite rules.

Case 1: Two apply steps. $(\sigma, B) \rightarrow (\text{apply}(e_1, \sigma), B \setminus \{e_1\}) = X_1$ and $(\sigma, B) \rightarrow (\text{apply}(e_2, \sigma), B \setminus \{e_2\}) = X_2$, where $e_1 \neq e_2$ are both enabled.

Since e_1 and e_2 are both enabled, their dependencies are disjoint from B . By Assumption 3.12, $e_1, e_2 \in E$, and since $e_1 \in B$ we have $e_1 \notin \text{deps}(e_2)$ (otherwise $\text{deps}(e_2) \cap B \neq \emptyset$, contradicting enabledness of e_2), and symmetrically $e_2 \notin \text{deps}(e_1)$. Therefore e_1 and e_2 are causally independent.

From X_1 : compensate to validity, obtaining $(\rho_R^*(\text{apply}(e_1, \sigma)), B \setminus \{e_1\})$; then apply e_2 (still enabled since its dependencies remain outside $B \setminus \{e_1\}$) and compensate, reaching

$$Z_1 = (\rho_R^*(\text{apply}(e_2, \rho_R^*(\text{apply}(e_1, \sigma)))), B \setminus \{e_1, e_2\}).$$

From X_2 : symmetrically, reach

$$Z_2 = (\rho_R^*(\text{apply}(e_1, \rho_R^*(\text{apply}(e_2, \sigma)))), B \setminus \{e_1, e_2\}).$$

By CC1, the state components of Z_1 and Z_2 are equal. Take $Z = Z_1 = Z_2$.

Case 2: Apply vs. compensation. $V_R(\sigma) = \perp$ and e is enabled in B . $(\sigma, B) \rightarrow (\text{apply}(e, \sigma), B \setminus \{e\}) = X_1$ and $(\sigma, B) \rightarrow (\rho_R(\sigma), B) = X_2$.

From X_1 : compensate to validity, reaching $(\rho_R^*(\text{apply}(e, \sigma)), B \setminus \{e\})$.

From $X_2 = (\rho_R(\sigma), B)$: apply e (still enabled, since compensation does not change B) to get $(\text{apply}(e, \rho_R(\sigma)), B \setminus \{e\})$; then compensate to validity, reaching $(\rho_R^*(\text{apply}(e, \rho_R(\sigma))), B \setminus \{e\})$.

By CC2, $\rho_R^*(\text{apply}(e, \sigma)) = \rho_R^*(\text{apply}(e, \rho_R(\sigma)))$. The buffer components are equal. Take $Z = (\rho_R^*(\text{apply}(e, \sigma)), B \setminus \{e\})$.

Case 3: Two compensation steps. Both rewrites apply ρ_R to the same state, producing the same successor $(\rho_R(\sigma), B)$. Take $Z = (\rho_R(\sigma), B)$. $\square$

5.3 Main Result

The following results are from [1]; proofs are included for self-containment.

Corollary 5.3 (Unique Normal Forms). *Under WFC and CC, every configuration (σ_0, E) has a unique normal form.*

Proof. By Lemma 5.1, $\mathcal{G}$ is terminating. By Lemma 5.2, $\mathcal{G}$ is locally confluent. By Newman’s Lemma [2], $\mathcal{G}$ is globally confluent. A terminating, confluent rewrite system has unique normal forms. $\square$

Remark 5.4 (Machine-checked). This result and the rest of the single-registry theory are mechanized in Coq/Rocq (checked with Rocq 9.3; CI also runs Coq 8.18 and 8.20). Newman’s Lemma is proved from scratch (no external libraries) as *strongly normalizing + locally confluent $\implies$ confluent*, and the Governance Rewrite System is formalized with its two rules; the lexicographic measure $(|B|, \Phi(\sigma))$ discharges termination and the three critical-pair cases (closed by CC1, CC2, and determinism of ρ_R) discharge local confluence, yielding confluence and unique normal forms. The gate script `coq/verify.sh` checks 669 named results, and `Print Assumptions` reports “Closed under the global context” (no axioms, no admitted lemmas) for every one. The results this paper relies on, by section:

- *Model and convergence.* ρ_R^* is constructed from WFC, as in Definition 3.15 (`base_def_rhostar`, `base_def_rhostar_least_unique`, `rho_star_canonical`; properties `rho_star_valid`, `rho_star_fix`, `rho_star_idem`), so Corollary 5.3 holds with WFC, CC1 on distinct, independent, co-enabled pairs, and CC2 as its only hypotheses, with the dependency-based enabledness of Definition 3.13 (`paper_causal_governance_unique_normal_forms`, `paper_governance_unique_normal_forms`; also `governance_unique_normal_forms`, `causal_governance_unique_normal_forms`). Termination and its bound: `base_lem_termination_bound`, attained by `lv_termination_bound_tight`; Lemma 3.8: `base_lem_finite_implies_ubc`; Remark 3.18: `base_rem_absorption`.
- *Stream level* (Definition 3.19, Theorem 5.5): `base_thm_convergence`, `stream_convergence`, `stream_agreement`, `base_rem_set_function`, `base_cor_quiescent`; counterexamples to the version-1 statements `base_thm_convergence_c_counterexample`, `base_thm_convergence_transient_counterexample`.
- *Domain independence* (the infinite-domains corollary of [1]): WFC over any well-founded order (`governance_wf_confluent`, `causal_governance_wf_confluent`, `base_cor_infinite`), with an instance on $\mathbb{Z}$ (`zw_confluent`, `zw_stream_agree`).

- *Tightness and exactness* (Section 6): `thm_necessity`, `ri_depth_exact`, `ri_no_ubic`, `ri_what_fails`, `ri_cc_any_extension`; `prop_cc_necessary`; the exact converses `cc_exact_from` (with ρ_R^* constructed: `wfc_cc_exact_from`) and `jc_exact`, and the qualifiers `masked_cc1`, `rho_star_qualifier`.
- *Worked example* (Section 4): `of_registry`, `of_cc1`, `of_cc2`, `of_paper_traces`, `of_processors`; the naive revert: `naive_cc1_fails`, `naive_cc2_fails`, `naive_stream_diverges`, `naive_paper_witness_fails`.
- *Complexity* (Theorem 7.1, model-level step bound): `base_thm_complexity_per_event`, `base_thm_complexity_comp_total`.
- *Calculus* (Section 10): `strong_absorption_iff_cc2`, `base_lem_repair_commute`, `base_lem_repair_idempotent`, the corrected footprint theorem and its parts (`calc_footprint_cc1_iff`, `calc_footprint_cc1_valid_iff`, `calc_footprint_cc1`, `calc_fp_absorb_iff_sa`, `calc_cc1_iff_commute_under_cc2`), the counterexamples to the version-1 statement (`base_thm_footprint_cc1_refuted`, `base_thm_footprint_cc1_raw_refuted`) and to dropping each hypothesis (`calc_valid_needs_commute`, `calc_valid_needs_disjoint`, `calc_valid_needs_repair_local`, `calc_all_needs_commute`, `calc_all_needs_absorb`, `calc_all_needs_repair_local`), the canonical-state pattern (`calc_canonical_pattern`, `base_pattern1_cc1_refuted`), and product lifting (`base_thm_product`, `base_thm_product_decomposable`).

Each abstract theorem comes with a concrete registry discharging all of its hypotheses with genuine nondeterminism (for example `example_confluent`, `calc_clamp_instance`, `calc_order_cc1`), so no theorem is vacuous. The soundness of the reference implementation’s own certification strategy is mechanized as well: events whose state-variable footprints (read and write sets) are disjoint commute (`disjoint_events_commute`; this concerns state variables and is a different statement from the invariant footprints of Section 10), and a repair strictly decreasing a potential is strongly normalizing (`repair_terminates`). The development is furthermore paired with two runnable checkers extracted from it, each proven axiom-free: one certifies that a compiled machine’s emitted step tables commute and stay in range (`check_tables_converges`), and one recomputes convergence directly from the machine’s rules by evaluating their expression trees, trusting neither the implementation’s enumeration nor its normalization (`checkBuild_converges`). Because the checking logic is extracted from the proof rather than reimplemented by hand, the two serve as independent differential oracles for the reference implementation: a defect in its hand-written verification cannot make a non-convergent machine pass either checker. The proof scripts are available in the `coq/` directory of the accompanying artifact; `coq/PAPER-MAP.md` maps every numbered result of this paper to its Coq statement. The federated results are listed in Remark 8.40.

Theorem 5.5 (Stream Convergence). *Let R be a registry satisfying WFC (Axiom 3.6) and CC (Axiom 3.16), and let P_1, P_2 be stream processors (Definition 3.19) consuming the same event stream S . Then:*

- **Validity:** *For each processor, after it completes the incremental compensation phase for any applied event, its state satisfies $V_R(\Psi_{P_j}(t)) = \top$.*
- **Order independence:** *Once a processor has finished processing its received event set, its state depends only on that set, not on the order in which events were applied.*
- **Agreement:** *Processors that have finished processing the same events are in the same state: if P_1 has finished processing $E_{P_1}(t_1)$, P_2 has finished processing $E_{P_2}(t_2)$, and $E_{P_1}(t_1) = E_{P_2}(t_2)$, then $\Psi_{P_1}(t_1) = \Psi_{P_2}(t_2)$. The times t_1 and t_2 may differ.*

Proof. We prove each part.

Part (a): Validity. By Lemma 5.1, compensation terminates. By Definition 3.19, the processor compensates after each event application until validity is restored. Therefore after completing the compensation phase for any applied event, $V_R(\Psi_{P_j}(t)) = \top$.

Part (b): Order independence. Each processor’s computation on event set $E = E_{P_j}(t)$ is a reduction sequence in the governance rewrite system from (σ_0, E) ; when the processor has finished, it has reached a normal form $(\sigma^*, \emptyset)$. Different causality-respecting orders correspond to different reduction sequences. By Corollary 5.3, all reduction sequences reach the same normal form. Therefore the final state σ^* depends only on E , not on the order.

Part (c): Agreement. If $E_{P_1}(t_1) = E_{P_2}(t_2) = E$ and both processors have finished, then both configurations are normal forms of reductions from (σ_0, E) . By Corollary 5.3, the normal form is unique, so $\Psi_{P_1}(t_1) = \Psi_{P_2}(t_2)$. $\square$

Remark 5.6 (Why “finished” is needed). Without the qualifier, part (c) fails: two processors can hold the same received set while one of them has not yet applied it. With a counter registry and the stream $\{e\}$, one processor that has applied e (state 1) and one that has received but not yet applied it (state 0, a zero-length reduction from $(0, \{e\})$) are both processors, both fair, and disagree. Eventual delivery alone also does not make received sets coincide on an infinite stream: if P_1 receives the n -th event at time n and P_2 at time $n + 1$, both can be finished at every time and still disagree at every time. What does hold is (c) as stated, at the same or at different times, and fixed-point agreement for finite streams (Corollary 5.8). Both counterexamples are mechanized (`base_thm_convergence_c_counterexample`, `base_thm_convergence_transient_counterexample`), as is the theorem (`base_thm_convergence_stream_agreement`).

Remark 5.7 (Semantic Projection). The theorem implies that the mapping $F(E) = \text{nf}(\sigma_0, E)$ —the unique normal form of the governance rewrite system starting from (σ_0, E) —is a well-defined function of the event set E , independent of causal linearization. This is the Church–Rosser property for our operational semantics.

Corollary 5.8 (Convergence Under Quiescent Streams). *If the stream $\mathcal{S}$ is finite and each processor is fair (it keeps processing while no new events arrive), then there exists a valid state σ^* such that both processors converge to σ^* : there exists T with $\Psi_{P_1}(t) = \Psi_{P_2}(t) = \sigma^*$ for all $t \geq T$.*

Proof. By eventual delivery, there exists T_N such that $E_{P_1}(t) = E_{P_2}(t) = \mathcal{S}$ for all $t \geq T_N$. From T_N on no new events arrive, so by fairness and Lemma 5.1 each processor finishes processing $\mathcal{S}$ at some time and stays at that normal form afterwards. By Theorem 5.5(c), from the later of the two finishing times T both processors are in the same valid state. $\square$

Remark 5.9 (Infinite Streams). For infinite streams, the received event sets grow monotonically toward $\mathcal{S}$ but may differ at every time, and eventual delivery does not make them coincide: a processor that is always one event behind another disagrees with it forever (Remark 5.6). The guarantee is Theorem 5.5(c): whenever two finished processors have received the same events, at the same or at different times, they hold the same state. The agreed-upon state may continue to change as new events arrive. Corollary 5.8 recovers fixed-point convergence when the stream is finite.

6 Necessity of the Structural Conditions

We show that UBC is necessary for uniform complexity, and that CC is the exact condition for agreement where an exact condition of this form exists.

6.1 Necessity of UBC

UBC is not merely convenient; it is necessary for a uniform bound on compensation depth, and thus for the constant per-event compensation overhead used in the complexity analysis (Section 7). Without it, compensation depth can grow without limit as event sets grow.

Theorem 6.1 (Unbounded Compensation Depth Without UBC). *For any $M \in \mathbb{N}$, there exists a registry R_∞ and a finite event set E such that some reduction sequence from (σ_0, E) in the governance rewrite system contains more than M compensation steps. Consequently, no uniform bound on compensation depth exists, and UBC fails.*

Proof. Define R_∞ with $\Sigma_{R_\infty} = \mathbb{Z}$, the single invariant $\psi(\sigma) = \top \iff \sigma = 0$, initial state $\sigma_0 = 0$, and compensation $\rho(\sigma) = \sigma + 1$ for $\sigma < 0$ and $\rho(\sigma) = \sigma - 1$ for $\sigma > 0$ (so compensation moves toward 0).

For each $n \in \mathbb{N}$, consider the single-event set $E_n = \{e_n\}$ where $\text{apply}(e_n, 0) = -n$. Under the processor model (Definition 3.19), the processor applies e_n to reach state $-n$, then compensates incrementally: $-n \rightarrow -(n-1) \rightarrow \dots \rightarrow -1 \rightarrow 0$. This requires exactly n compensation steps.

For any proposed bound M , choose $n = M + 1$. The compensation episode after this single event contains $n > M$ steps. Therefore no finite M uniformly bounds compensation depth, and UBC fails.

The measure $\Phi(\sigma) = |\sigma|$ strictly decreases under ρ , so WFC holds and compensation terminates for each fixed E_n . The pathology is not non-termination but *unbounded* termination: the compensation depth required to restore validity grows without limit. $\square$

Remark 6.2 (What Fails). The counterexample satisfies WFC (compensation terminates for every fixed E_n) but violates UBC (compensation depth is n , unbounded). In fact every reduction from $(0, \{e_n\})$ to a normal form has exactly n compensation steps, and every measure witnessing WFC for R_∞ is unbounded, not only $|\sigma|$. CC also holds: the only valid state is 0, so $\rho^*(\sigma) = 0$ for every σ , and both sides of CC1 and of CC2 equal 0 for every state and every pair of events, whatever $\text{apply}(e_n, -)$ does away from 0 (take, for instance, $\text{apply}(e_n, \sigma) = \sigma - n$). The pathology is purely one of unbounded depth, which invalidates the uniform step bound and the constant per-event compensation cost in Theorem 7.1. Finite semantic state spaces prevent this by bounding Φ (Lemma 3.8).

6.2 Necessity of CC

Without CC, processors applying the same events in different causality-respecting orders can reach different valid states (Proposition 6.3), and under free delivery this is the only way agreement can fail (Theorem 6.5).

Proposition 6.3 (Disagreement Without CC). *There exists a registry satisfying WFC and UBC, and a finite event set E with two causality-respecting orderings, such that the governance rewrite system reaches different valid normal forms under the two orderings.*

Proof. Let $\Sigma_R = \{A, B, C, D\}$ with valid states $\{A, B\}$ and compensation $\rho_R(C) = A$, $\rho_R(D) = B$. UBC holds with $M = 1$.

Define two events e_1, e_2 with $\text{deps}(e_1) = \text{deps}(e_2) = \emptyset$, so both are enabled from the initial configuration $(A, \{e_1, e_2\})$ and are causally independent under Definition 3.11. Their application

is:

	A	B	C	D
e_1	C	C	C	D
e_2	D	C	C	D

Evaluate CC1 at $\sigma = A$. Under the processor model (Definition 3.19), each processor normalizes after each application, so the comparison is between $\rho_R^*(A_{e_2}(\rho_R^*(A_{e_1}(A))))$ and the symmetric order.

Path 1 (e_1 then e_2): $\text{apply}(e_1, A) = C$, $\rho_R^*(C) = A$, $\text{apply}(e_2, A) = D$, $\rho_R^*(D) = B$. Normal form: B .

Path 2 (e_2 then e_1): $\text{apply}(e_2, A) = D$, $\rho_R^*(D) = B$, $\text{apply}(e_1, B) = C$, $\rho_R^*(C) = A$. Normal form: A .

Since $A \neq B$, CC1 is violated. Two processors consuming $\{e_1, e_2\}$ in different causality-respecting orders reach different valid states despite WFC and UBC holding. $\square$

Remark 6.4. The key feature is that ρ_R maps distinct invalid states to *distinct* valid states ($C \mapsto A$, $D \mapsto B$), creating path-dependent repair. Contrast this with the order fulfillment example (Section 4), where compensation preserves approval intent via the held state, ensuring path-independent repair. Theorem 6.5 makes precise in what sense CC captures the boundary between these cases.

Proposition 6.3 is existential: it shows that *some* failure of CC produces disagreement. The converse holds in the following exact form. Say that *every buffered event may fire* (free delivery) when an event is enabled as soon as it is in the buffer, as for events without causal dependencies, and that compensation is *canonical* when ρ_R^* returns a valid state, as Definition 3.15 guarantees under WFC. A state is *reachable* from σ_0 if some reduction from some (σ_0, E) passes through it.

Theorem 6.5 (CC Is Exact). *Let R satisfy WFC, with ρ_R^* as in Definition 3.15, and let every buffered event fire freely. Then every configuration (σ_0, E) with E finite has a unique normal form if and only if CC1 and CC2 hold at every state reachable from σ_0 . Under an arbitrary enabledness predicate (causal or guarded), the system is confluent from a configuration c_0 if and only if, at every configuration reachable from c_0 , the governed successors of every critical pair are joinable; CC1 and CC2 are the special case in which the join is an equality.*

Proof sketch. Sufficiency is Corollary 5.3, with Newman’s Lemma localized to the step-closed set of reachable configurations. For necessity under free delivery, the two sides of CC1 at σ are the normal forms of the runs e_1 then e_2 and e_2 then e_1 from $(\sigma, \{e_1, e_2\})$, and the two sides of CC2 at an invalid σ are the normal forms of apply-then-compensate and compensate-then-apply from $(\sigma, \{e\})$. A reachable σ is reached from σ_0 by delivering some word w first, so a failure of CC at σ yields two distinct normal forms of $(\sigma_0, w \cdot e_1 e_2)$ or $(\sigma_0, w \cdot e)$. The general case is Newman’s Lemma together with the observation that a non-joinable critical pair at a reachable configuration is a pair of distinct normal forms. Both directions are mechanized (`cc_exact_from`, `wfc_cc_exact_from`, `jc_exact`). $\square$

Remark 6.6 (Scope of the Converse). Both qualifiers are needed. If ρ_R^* is only required to be some compensation reduct of its argument (the identity qualifies), CC1 can fail while every configuration has a unique normal form. And under causal enabledness, CC1 can fail for a co-enabled pair at a reachable configuration while every run still reaches the same normal form: events M_1 (write 1) and M_2 (write 2) do not commute, but a reset event that depends on both erases the difference, so from $(0, \{M_1, M_2, M_{\text{reset}}\})$ every normal form is $(0, \emptyset)$ (`rho_star_`

qualifier, masked_cc1). There the exact condition is the joinability form of Theorem 6.5, not CC1. Together with Theorem 6.1, which satisfies CC and violates UBC, and Proposition 6.3, which satisfies WFC and UBC and violates CC, this shows each condition is needed for its guarantee: UBC for uniform step bounds, CC (exactly, under free delivery) for agreement.

7 Complexity Analysis

Theorem 7.1 (Convergence Complexity). *For a stream prefix of length n under a registry satisfying UBC with bound M and $k = |I_R|$ invariants, the model-level cost of registry-governed stream processing is $O(n)$ for event application, validation, and compensation combined. If causality maintenance is implemented with a priority queue, the total cost is $O(n \log n)$, where k and M are treated as constants of system design.*

Proof. We analyze the per-event cost, separating model-level costs (which follow from the formalism) from scheduling costs (which depend on implementation).

Model-level per-event costs. For each event e_i ($1 \leq i \leq n$):

- *Event application:* $O(1)$ (state update).
- *Invariant validation:* $O(k) = O(1)$ (evaluate each ψ_j).
- *Compensation:* At most M steps, each $O(1)$. Since M is bounded by UBC, this is $O(1)$.

Registry governance therefore adds $O(1)$ overhead per event beyond what the base stream processor already performs.

Scheduling overhead. Maintaining a set of enabled events and resolving causal dependencies requires implementation-dependent data structures. With a priority queue keyed by timestamp and identifier, each event insertion and dependency-resolution check costs $O(\log i)$, yielding:

Total cost.

$$T(n) = \sum_{i=1}^n (\underbrace{O(1)}_{\text{model}} + \underbrace{O(\log i)}_{\text{scheduling}}) = O(n) + O\left(\sum_{i=1}^n \log i\right) = O(n \log n).$$

The dominant term is causality maintenance. Validation and compensation contribute no asymptotic overhead. Systems with native causal ordering (e.g., single-partition Kafka topics) eliminate the $O(\log i)$ term, reducing the bound to $O(n)$. $\square$

8 Federated Convergence

The convergence theorem (Theorem 5.5) applies within a single registry. The product composition theorem (Theorem 10.15) extends it to independent registries with disjoint invariants. But real multi-registry systems have *dependencies* across organizational boundaries: a manufacturer’s specifications constrain supplier capabilities; a regulator’s rules constrain bank behavior. We now extend convergence to registries connected by morphisms encoding such constraints.

The central observation is that a federation of registries connected by morphisms *is* a registry in the sense of Section 3: the federated state space is the product of component state spaces, morphism consistency conditions are additional invariants, and morphism repair is additional

compensation. Federated convergence then follows from the single-registry theory if the federated registry satisfies WFC and CC. WFC, and with it a unique repair normal form, is derived from the network's structure (Lemma 8.10). CC is not: a target event can read a shared value that morphism repair overwrites, and then the components' own CC does not carry over to the federation (Remark 8.8). We isolate the two per-edge conditions that are needed, C1 and C2 (Definition 8.7), prove that they give federated CC and convergence (Theorems 8.11 and 8.12), and prove that their reachable forms are exact.

8.1 Registry Morphisms

Definition 8.1 (Registry Morphism). Given registries R_A and R_B , a *registry morphism* $\phi : R_A \rightarrow R_B$ consists of a *decomposition* of B 's state space—an isomorphism $\langle \pi_B^{\text{sh}}, \pi_B^{\text{loc}} \rangle : \Sigma_B \xrightarrow{\sim} S_B \times L_B$ with reconstruction $\text{merge}_B : S_B \times L_B \rightarrow \Sigma_B$ satisfying $\text{merge}_B(\pi_B^{\text{sh}}(\sigma), \pi_B^{\text{loc}}(\sigma)) = \sigma$ —and a function $\phi : \Sigma_A \rightarrow S_B$ mapping source states to *shared* target states. Write $\sigma_B[\text{sh} \mapsto s] := \text{merge}_B(s, \pi_B^{\text{loc}}(\sigma_B))$.

The morphism satisfies:

(M1) **Validity preservation under overwrite:** If $V_A(\sigma_A) = \top$ and $V_B(\sigma_B) = \top$, then $V_B(\sigma_B[\text{sh} \mapsto \phi(\sigma_A)]) = \top$.

When $\pi_B^{\text{sh}} = \text{id}$ (L_B trivial), the morphism constrains B completely (*total morphism*).

Remark 8.2. M1 is not standard function-level validity preservation; it is validity preservation under shared-component overwrite. A sufficient condition: if V_B factors as $V_B(\sigma) \iff V_B^{\text{sh}}(\pi_B^{\text{sh}}(\sigma)) \wedge V_B^{\text{loc}}(\pi_B^{\text{loc}}(\sigma))$ (shared and local validity independent), then M1 reduces to $V_A(\sigma_A) = \top \Rightarrow V_B^{\text{sh}}(\phi(\sigma_A)) = \top$. The shared/local decomposition models partial authority: a manufacturer constrains product specifications (shared) but not supplier pricing (local).

M1 is a nonvacuous constraint: its antecedent requires both a valid source and a valid target state, and WFC (Axiom 3.6) guarantees every component registry has a nonempty valid set, since iterated compensation reaches a valid state from any state. A registry whose invariants admit no valid state is therefore excluded at the component level, not silently satisfying M1 here.

Definition 8.3 (Tree-Shaped Morphism Network). A *morphism network* $\mathcal{N} = (\{R_i\}_{i \in \mathcal{I}}, \{\phi_{ij}\}_{(i,j) \in E})$ is *tree-shaped* if $(\mathcal{I}, E)$ is a directed forest: each connected component is a tree rooted at a *source* registry (no incoming morphisms), and each non-source has exactly one incoming edge.

8.2 Federated Construction

Definition 8.4 (Federated Registry). Given a tree-shaped morphism network $\mathcal{N}$, the federated registry R_{Fed} has state space $\Sigma_{\text{Fed}} = \prod_i \Sigma_{R_i}$ with projections $\pi_i(\bar{\sigma}) = \sigma_i$. Invariants comprise: (i) *local invariants*—each $\psi_j \in I_{R_i}$ lifted to the product as $\hat{\psi}_{i,j}(\bar{\sigma}) = \psi_j(\sigma_i)$ —and (ii) *morphism invariants*—for each $(i, j) \in E$, $\mu_{ij}(\bar{\sigma}) = \top \iff \phi_{ij}(\sigma_i) = \pi_j^{\text{sh}}(\sigma_j)$.

Definition 8.5 (Federated Normalization Operator). Let $i_1, \dots, i_n$ be a topological ordering of $\mathcal{I}$. The operator $\rho_{\text{Fed}} : \Sigma_{\text{Fed}} \rightarrow \Sigma_{\text{Fed}}$ acts in two phases:

Phase 1 (Local normalization): For each i , replace σ_i with $\rho_{R_i}^*(\sigma_i)$.

Phase 2 (Morphism repair): Process edges in topological order. For $(i, j) \in E$, if $\phi_{ij}(\sigma_i) \neq \pi_j^{\text{sh}}(\sigma_j)$, set $\sigma_j \mapsto \sigma_j[\text{sh} \mapsto \phi_{ij}(\sigma_i)]$.

Local normalization precedes morphism repair because M1 guarantees that overwriting the shared component of a locally valid target with the morphism image of a locally valid source preserves target validity.

Definition 8.6 (Federated Governance Rewrite System). A *federated event* e targets one registry $R_{m(e)}$: applying it replaces $\sigma_{m(e)}$ by $\text{apply}(e, \sigma_{m(e)})$ and leaves every other component unchanged, so cross-registry effects arise only through morphism violation and repair. Write V_{Fed} for the conjunction of all local and morphism invariants (*federal validity*). The *federated governance rewrite system* $\mathcal{G}_{\text{Fed}}$ operates on configurations $(\bar{\sigma}, B)$ as in Definition 3.14, with two rules:

1. **Compensation step:** $(\bar{\sigma}, B) \rightarrow (\rho_{\text{Fed}}(\bar{\sigma}), B)$ whenever $V_{\text{Fed}}(\bar{\sigma}) = \perp$.
2. **Guarded apply step:** $(\bar{\sigma}, B) \rightarrow (\text{apply}(e, \bar{\sigma}), B \setminus \{e\})$ whenever $e \in B$ is enabled and $V_{\text{Fed}}(\bar{\sigma}) = \top$.

The guard says that events are applied to normalized states: a federated processor compensates, then applies the next event. An apply step followed by the compensation it triggers is the *governed step* $\bar{\sigma} \mapsto \rho_{\text{Fed}}(\text{apply}(e, \bar{\sigma}))$. The unguarded system, in which apply steps may also fire at federally invalid states, is discussed in Remark 8.14.

Notation. For an event e of registry j write $\text{sig}_e := \rho_{R_j}^* \circ \text{apply}(e, -)$ (the component step: the event, then component compensation). For a registry j and a family $z = (z_i)_{i \in S(j)}$ of states of its sources write $\text{ow}_z(x) := x[\text{sh} \mapsto \phi_{ij}(z_i)]$ (the morphism repair of j against z ; for a multi-source target the resolved value $\Gamma_j(z)$ of Section 8.5 replaces $\phi_{ij}(z_i)$; for a source registry ow_z is the identity). A state b of j is *consistent with* z if $b = \text{ow}_z(b)$. Two federated events are *federated-independent* if they target different registries, or target the same registry and are causally independent.

Definition 8.7 (Cross-Registry Compensation Commutativity). A morphism network satisfies:

- (C1) **Cross-registry CC.** For every event e of registry j , all valid source families z, z' of j , and every valid state b of j consistent with z' :

$$\text{ow}_z(\text{sig}_e(\text{ow}_z(b))) = \text{ow}_z(\text{sig}_e(b)).$$

- (C2) **Repaired CC.** For all causally independent events e_1, e_2 of the same registry j , every valid source family z of j , and every valid b consistent with z :

$$\text{ow}_z(\text{sig}_{e_2}(\text{ow}_z(\text{sig}_{e_1}(b)))) = \text{ow}_z(\text{sig}_{e_1}(\text{ow}_z(\text{sig}_{e_2}(b)))).$$

C1 says that the outcome of a target event, after repair against the current sources z , does not depend on whether the sources moved from z' to z before or after the event. The event may read and write the shared component; the final overwrite appears on both sides. C2 is CC for the step “event, then repair”. For a source registry (ow the identity) C1 is trivial and C2 is exactly the registry’s own CC1 at valid states. Both quantify over one edge (a target and its sources), so on finite state spaces both are checked by enumeration, like M1.

Remark 8.8 (Why component CC is not enough). For a target, neither C1 nor C2 follows from the target’s own CC and M1. In a two-registry tree satisfying M1 and component WFC and CC, a target event that copies a shared flag (which the morphism overwrites) into a local counter ends with the counter at 0 or at 1 depending on whether a source event is propagated first: C1 fails and two event orders diverge (Coq: `audit_counterexample`, and against this paper’s exact hypotheses `fed_thm_fed_convergence_refuted`). A second tree satisfies C1 and component CC (the target’s two events commute locally) but violates C2, because the repair between the two events erases what the first wrote into a shared variable; it diverges as well (`c2_counterexample`, `fed_c2_paper_counterexample`). So neither condition can be dropped.

8.3 The Authority Argument

In a directed morphism $\phi : R_A \rightarrow R_B$, registry A is *authoritative* over B 's shared component: in a federally valid state, B 's shared state is completely determined by A 's state. Throughout, **(H)** denotes the standing hypotheses: a tree-shaped morphism network (Definition 8.3) whose morphisms satisfy M1 and whose component registries satisfy WFC and CC.

Lemma 8.9 (Authority Determination). *Assume (H).*

- (a) *If $(\bar{\sigma}^0, B) \rightarrow^* (\bar{\nu}, \emptyset)$ in $\mathcal{G}_{\text{Fed}}$ with $\bar{\nu}$ federally valid, then for every source registry i , ν_i is the unique normal form of the single-registry rewrite system of R_i from $(\sigma_i^0, B|_i)$, where $B|_i$ is the set of events of B targeting i (events targeting other registries and morphism repair do not modify source components).*
- (b) *In every federally valid $\bar{\sigma}$, the shared component of a non-source j with incoming edge from i satisfies $\pi_j^{\text{sh}}(\sigma_j) = \phi_{ij}(\sigma_i)$: it is a function of the source state alone.*
- (c) *Morphism repair never modifies π_j^{loc} .*

The local component of a target is not determined by component CC: two event orders can leave it different (Remark 8.8). Under C1 and C2 it is determined by Theorem 8.12 and computed by Corollary 8.15. (Coq: (a) `fed_lem_authority_a`, with instance `fed_lem_authority_a_instance`; (b) `fed_lem_authority_b`; (c) `fed_lem_authority_c_local`; the version 1 convergence clause is refuted by `fed_lem_authority_c_refuted`. Part (a) is proved for the unguarded system and holds for $\mathcal{G}_{\text{Fed}}$, whose reductions are among the unguarded ones.)

Lemma 8.10 (Single-Round Termination). *Assume (H). For every federated state $\bar{\sigma}$, not only locally valid ones, $\rho_{\text{Fed}}(\bar{\sigma})$ is federally valid. Phase 1 on registry j takes at most $\Phi_j(\sigma_j)$ compensation steps (at most M_j under UBC); Phase 2 processes each edge once (unique target: each registry's shared component is set once and not revisited). Morphism repair cannot reintroduce local invalidity: M1 guarantees that overwriting the shared component of a locally valid target with the morphism image of a locally valid source preserves target validity. Since Phase 1 establishes local validity before Phase 2 begins, and Phase 2 only modifies shared components, every intermediate state of Phase 2 is locally valid. Moreover ρ_{Fed} is the identity on federally valid states, so it is an idempotent retraction onto them, and its result does not depend on the topological order chosen.*

(Coq: `fed_lem_fed_termination`, `fed_phase1_bound`, `single_round`, `phase2_invariant`, `grho_valid`, `grho_id`; order independence and the retraction on the categorical paper's consistent set: `cat_thm_one_idempotent`, `cat_thm_one_image`, `cat_thm_one_order_independent`.) Repair confluence therefore composes with no condition beyond M1: the *repair normal form* $\rho_{\text{Fed}}(\bar{\sigma})$ of any state is unique. Event confluence is the subject of the next two theorems.

Theorem 8.11 (Federated CC). *Assume (H), C1 and C2. Then the governed step satisfies CC1 at every federally valid state: for every federally valid $\bar{\sigma}$ and all federated-independent events e_1, e_2 ,*

$$\rho_{\text{Fed}}(\text{apply}(e_2, \rho_{\text{Fed}}(\text{apply}(e_1, \bar{\sigma})))) = \rho_{\text{Fed}}(\text{apply}(e_1, \rho_{\text{Fed}}(\text{apply}(e_2, \bar{\sigma})))).$$

Proof. Let $\bar{\sigma}$ be federally valid and write gov_e for the governed step. On a federally valid state, gov_e replaces the component $m(e)$ by sig_e of it and then re-repairs every target in topological order against its finalized sources; by Lemma 8.9(b) every shared component is the image of its sources. Process the registries in topological order and compare the two sides component by component. A registry that is neither targeted by e_1 or e_2 nor downstream of one is unchanged

on both sides. If e_1, e_2 target different registries $m_1 \neq m_2$, the component m_1 is, on one side, ow_z applied to sig_{e_1} of its consistent state and, on the other, the same after its sources moved from z' to z through e_2 ; C1 at (e_1, z, z', b) says the two agree, and symmetrically for m_2 . Every other target is the repair of its own (unchanged) state against sources that agree by induction. If e_1, e_2 target the same registry, the two sides of C2 at the registry's consistent state are exactly its two final values, and the downstream registries again agree by induction. The formal proof is `fed_events_commute`, and against (H) `fed_thm_fed_cc_corrected_machine`. $\square$ $\square$

CC2 is not needed for $\mathcal{G}_{\text{Fed}}$: no apply step fires at a federally invalid state, so the critical pair “apply versus compensation” does not arise. Under (H), C1 and C2 do not give CC2 of the unguarded system (Remark 8.14). In the guarded proof the components' own CC enters only through C2, which for a source registry is that registry's CC1.

Theorem 8.12 (Federated Normalization Confluence). *Assume (H) and let $\bar{\sigma}^0$ be federally valid.*

- (a) *If C1 and C2 hold, every configuration $(\bar{\sigma}^0, B)$ has a unique normal form in $\mathcal{G}_{\text{Fed}}$, and it is federally valid: any two processors that consume the same federated events from $\bar{\sigma}^0$, applying each event to a normalized state, converge to the same federally valid state.*
- (b) **(Exact condition.)** *Every configuration $(\bar{\sigma}^0, B)$ has a unique normal form in $\mathcal{G}_{\text{Fed}}$ if and only if C1 holds at every reachable witness $(e, z, z', b) = (e, \text{gov}_a(\bar{\sigma}), \bar{\sigma}, \sigma_{m(e)})$ and C2 at every reachable witness $(e_1, e_2, z, b) = (e_1, e_2, \bar{\sigma}, \sigma_{m(e_1)})$, where $\bar{\sigma}$ ranges over the states reachable from $\bar{\sigma}^0$ by governed steps and a over events of registries other than $m(e)$. Equivalently, federated-independent events commute after re-normalization at every reachable state (the global condition GC).*

Proof. Termination. By Lemma 8.10, ρ_{Fed} maps any federated state to a federally valid one in a single application, so the federated measure $\Phi_{\text{Fed}}(\bar{\sigma}) = \mathbf{1}[V_{\text{Fed}}(\bar{\sigma}) = \perp] \in \{0, 1\}$ satisfies WFC, and the lexicographic order $(|B|, \Phi_{\text{Fed}})$ gives termination as in Lemma 5.1.

(a). Local confluence of $\mathcal{G}_{\text{Fed}}$: two compensation steps coincide (ρ_{Fed} is deterministic); an apply step and a compensation step are never enabled together (the guard); two apply steps at a federally valid state close after one compensation each by Theorem 8.11. Newman's Lemma gives unique normal forms, which are federally valid by the definition of normal form.

(b). Every reachable state of $\mathcal{G}_{\text{Fed}}$ that is federally valid is the result of a sequence of governed steps from $\bar{\sigma}^0$. At such a state the two sides of C1 at the reachable witness are the registry $m(e)$ after the governed runs $e; a$ and $a; e$ (which differ by one allowed swap), and the two sides of C2 are the registry after $e_1; e_2$ and $e_2; e_1$. So a failing witness is an observable divergence, and conversely, if every reachable witness passes, independent events commute at every reachable state and the argument of (a) applies with “reachable” in place of “every”. $\square$ $\square$

(Coq: (a) `fed_thm_fed_convergence_guarded`, in the machine model `fed_interleavings_converge` and `fed_permutations_converge`; (b) `fed_thm_fed_convergence_exact`, `fed_guarded_exact`, `fed_exact`; with any number of intervening events in place of one, `fed_exact_full`; GC: `acyclic_gc_iff`, `gc_iff_reach`; static C1 and C2 imply the reachable forms: `static_c1_c2_gc`; the two divergence constructions: `conv_c1_diverge`, `conv_c2_diverge`. Non-vacuity: a supply federation satisfying (H), C1 and C2, `fed_supply_paper_instance`.) Coq's buffers carry no causal dependencies (every buffered event is enabled); with dependencies the reductions are a subset, so (a) carries over.

Remark 8.13 (Static and reachable checks). Static C1 and C2 (Definition 8.7) quantify over all valid source families and all valid consistent target states; they are per-edge and are what a

build-time verifier enumerates. Their reachable forms in Theorem 8.12(b) are exact but require exploring runs. The naive converse “C1 fails, so some two interleavings diverge” is false: a federation can violate C1 at a pair of source states that no run moves between, while every interleaving converges (`naive_converse_fails`).

Remark 8.14 (The unguarded system). Version 1 of this paper used the unguarded system, in which an apply step may fire at a federally invalid state. There C1 and C2 are not sufficient: a tree satisfying (H), C1 and C2, on which every permutation of governed steps converges and $\mathcal{G}_{\text{Fed}}$ has unique normal forms, nevertheless reaches two normal forms in the unguarded system from one federally valid start, because two target events are applied before compensation and the second reads the shared value the first wrote (`fed_grs_c1_c2_insufficient`). For the unguarded system the exact condition is CC1 and CC2 of the governed step on the states reachable from the start (`fed_grs_exact`, an instance of the single-registry exact converse), which implies the reachable C1/C2 condition but not conversely (`fed_grs_un_c1_c2`). A clean sufficient condition is **XU**: C1 for *every* valid target state b , not only consistent ones. Under (H) and XU the unguarded system satisfies CC1 and CC2 at every state and has unique normal forms (`fed_thm_fed_cc_corrected`, `fed_thm_fed_convergence_corrected`, `grs_unique_nf`). XU with component CC implies C1 and C2 (`xu_implies_c1_c2`), and XU is also what a distributed model with explicit propagation steps needs (`dist_interleavings_converge`).

Corollary 8.15 (Constructive Normal Form). *Assume (H), C1 and C2, and let $\bar{\sigma}$ be federally valid. For every event sequence, the federated normal form $\bar{Z}$ is the unique solution, computed registry by registry in topological order, of the following equations. For registry j with events $e_1, \dots, e_m$ (in sequence order),*

$$x_0 = \text{ow}_Z(\sigma_j), \quad x_t = \text{ow}_Z(\text{sig}_{e_t}(x_{t-1})), \quad Z_j = \text{ow}_Z(x_m),$$

where ow_Z reads only the finalized sources of j . So: compute each source’s single-registry run on its own events (ow is the identity there), then, in topological order, run each target’s events with the repair against its finalized sources interleaved after every event. The product state space is never materialized.

(Coq: `fed_cor_fed_nf_corrected`, `fed_nf_constructive`.) Under XU the interleaved repairs can be dropped and the recipe of version 1 holds: run the target’s events as a single registry, then overwrite its shared component once (`fed_nf_recipe_xu`). Under C1 and C2 alone that recipe is false (`fed_cor_fed_nf_refuted`: it gives a different state from every event order).

Remark 8.16 (Compensation Compatibility is Derived). One might expect morphisms to satisfy $\phi(\rho_A^*(\sigma)) = \rho_B^*(\phi(\sigma))$. The authority argument shows this is unnecessary: ρ_{Fed} normalizes locally first, then projects via ϕ . The composition “normalize source, then project” produces a valid shared component by M1 alone. We never need the reverse.

Remark 8.17 (Relationship to Product Composition). Theorem 10.15 handles the special case $E = \emptyset$: independent registries with no morphism constraints. There C1 is trivial and C2 is each component’s own CC1, so Theorem 8.12 reduces to product composition. Federated convergence generalizes it to registries connected by directed semantic constraints.

8.4 Necessity of the Structural Conditions

Proposition 8.18 (Necessity of Acyclicity). *There exists a cyclic morphism network where each component satisfies WFC and CC, all morphisms satisfy M1, but federated compensation cycles.*

Proof. $R_A = R_B$: $\Sigma = \{0, 1\}$, both valid, trivial compensation. Morphisms: $\phi_{AB}(x) = 1 - x$ (flip), $\phi_{BA} = \text{id}$. Both satisfy M1. From $(0, 0)$: repair on ϕ_{AB} gives $(0, 1)$; repair on ϕ_{BA} gives $(1, 1)$; repair on ϕ_{AB} gives $(1, 0)$; repair on ϕ_{BA} gives $(0, 0)$. Cycle. Every repair step is forced (each state violates exactly one morphism invariant), no state is federally valid, and compensation never terminates from any state (Coq: `prop_cycle_necessary`, `cycle_paper_trace`). $\square$ $\square$

Proposition 8.19 (Necessity of M1). *There exists an acyclic network where each component satisfies WFC and CC, but a morphism violates M1, and federated compensation oscillates.*

Proof. R_A : $\Sigma_A = \{0\}$, trivial. R_B : $\Sigma_B = \{0, 1\}$, $V_B(0) = \top$, $V_B(1) = \perp$, $\rho_B(1) = 0$. Total morphism $\phi_{AB}(0) = 1$ (violates M1). From $(0, 0)$: morphism repair sets $\sigma_B = 1$; local compensation sets $\sigma_B = 0$; morphism repair sets $\sigma_B = 1$. Oscillation between morphism repair and local compensation: no state is federally valid and compensation never terminates (Coq: `prop_m1_necessary`, `m1_paper_trace`). In particular one application of ρ_{Fed} from $(0, 0)$ returns $(0, 1)$, which is not federally valid, so Lemma 8.10 needs M1 (`m1_single_round_fails`). $\square$ $\square$

Remark 8.8 gives the corresponding counterexamples for C1 and C2: with acyclicity, M1 and component WFC and CC all holding, dropping either check lets two event orders reach different federally valid states.

Remark 8.20 (The Multi-Source Boundary). The tree-shaped restriction excludes networks where a registry has multiple incoming morphisms from independent sources. When two morphisms ϕ_{ij} and ϕ_{kj} both target registry j , a federally valid state requires $\phi_{ij}(\sigma_i) = \phi_{kj}(\sigma_k) = \pi_j^{\text{sh}}(\sigma_j)$ —constraining the *sources* to produce compatible projections. The authority argument breaks: no single source determines the target. Resolving this requires additional machinery (composition consistency along common ancestors, target agreement predicates, or conflict resolution operators). We develop the last of these, conflict resolution operators, in Section 8.5, recovering the unique repair normal form on any acyclic network and, under C1 and C2, event convergence.

8.5 Multi-Source Resolution

Remark 8.20 identified the obstruction to networks in which a registry has several incoming morphisms: no single source determines the target’s shared component. We remove the tree restriction—admitting any acyclic network—by replacing the per-edge authority function at a multi-source target with a single *resolution operator*. The authority argument then generalizes: a target’s shared component becomes a deterministic function not of one source but of all of them.

Definition 8.21 (Resolution Operator). Let j be a registry with source set $S(j) = \{i : (i, j) \in E\}$. A *resolution operator* for j is a map

$$\Gamma_j : \prod_{i \in S(j)} \Sigma_{R_i} \longrightarrow S_j$$

from the source states to j ’s shared component, satisfying:

- (R1) **Source determinacy**: Γ_j is a function of the source states alone; it does not read $\pi_j^{\text{loc}}(\sigma_j)$.
- (R2) **Validity preservation**: for all $(\sigma_i)_{i \in S(j)}$ with each $V_{R_i}(\sigma_i) = \top$ and all σ_j with $V_{R_j}(\sigma_j) = \top$,

$$V_{R_j}(\sigma_j[\text{sh} \mapsto \Gamma_j((\sigma_i)_{i \in S(j)})]) = \top.$$

When $|S(j)| = 1$, taking $\Gamma_j(\sigma_i) = \phi_{ij}(\sigma_i)$ recovers the authority morphism and (R2) is exactly M1. A resolution operator thus strictly generalizes the single-source morphism image: it expresses merges—priority, conjunction, most-restrictive-wins—that no single authority can. As with M1, (R2) is nonvacuous: WFC (Axiom 3.6) on each source and on j makes both antecedents satisfiable.

Definition 8.22 (Resolved Network). A *resolved network* is an acyclic morphism network in which every target with $|S(j)| > 1$ carries a resolution operator Γ_j ; single-source targets retain their authority morphism. The federated registry is as in Definition 8.4, with the morphism invariants for a multi-source target replaced by the *resolution invariant*

$$\mu_j(\bar{\sigma}) = \top \iff \pi_j^{\text{sh}}(\sigma_j) = \Gamma_j((\sigma_i)_{i \in S(j)}).$$

The normalization operator ρ_{Fed} carries over from Definition 8.5 unchanged, with Phase 2 setting each multi-source target's shared component in a single step: for target j in topological order, if $\pi_j^{\text{sh}}(\sigma_j) \neq \Gamma_j((\sigma_i)_{i \in S(j)})$, set $\sigma_j \mapsto \sigma_j[\text{sh} \mapsto \Gamma_j((\sigma_i)_{i \in S(j)})]$. Acyclicity guarantees a topological ordering in which every source $i \in S(j)$ precedes j .

Lemma 8.23 (Single-Round Termination, Resolved). *On a resolved network whose single-source morphisms satisfy M1, whose resolution operators satisfy (R1)–(R2), and whose components satisfy WFC and CC, a single application of ρ_{Fed} to any federated state produces a federally valid state.*

Proof. Identical to Lemma 8.10, with the single-source repair $\phi_{ij}(\sigma_i)$ replaced by $\Gamma_j((\sigma_i)_{i \in S(j)})$ and M1 by (R2). Phase 1 establishes local validity; Phase 2 sets each target's shared component exactly once (its resolution operator consumes all incoming edges in one evaluation, and sources are finalized first by acyclicity). By (R2), overwriting a locally valid target's shared component with the resolved value of locally valid sources preserves target validity, so Phase 2 never reintroduces invalidity. (Coq: `fed_lem_resolved_termination`.) $\square$ $\square$

The rewrite system $\mathcal{G}_{\text{Fed}}$ (Definition 8.6), the notation ow_z (with $\Gamma_j(z)$ as the resolved value) and the checks C1 and C2 (Definition 8.7) carry over unchanged. Write $(\mathbf{H}_R)$ for the hypotheses of Lemma 8.23: an acyclic resolved network, M1 on single-source morphisms, (R1)–(R2) on resolvers, and WFC and CC on every component. Lemma 8.9 holds under $(\mathbf{H}_R)$ with $\Gamma_j((\sigma_i)_{i \in S(j)})$ in place of $\phi_{ij}(\sigma_i)$ in (b) (the Coq statements are proved for resolved networks, the tree case being an instance).

Theorem 8.24 (Federated Convergence with Resolution). *Assume $(\mathbf{H}_R)$ and let $\bar{\sigma}^0$ be federally valid.*

- (a) *If C1 and C2 hold, every configuration $(\bar{\sigma}^0, B)$ has a unique normal form in $\mathcal{G}_{\text{Fed}}$, and it is federally valid: any two processors consuming the same federated events from $\bar{\sigma}^0$ converge to the same federally valid state.*
- (b) *Every configuration $(\bar{\sigma}^0, B)$ has a unique normal form in $\mathcal{G}_{\text{Fed}}$ if and only if C1 and C2 hold at the reachable witnesses of Theorem 8.12(b).*

Proof. As for Theorem 8.12. Lemma 8.23 gives WFC with the trivial measure $\Phi_{\text{Fed}} \in \{0, 1\}$. The proof of Theorem 8.11 uses only that every target is repaired once, in topological order, against its finalized sources, by a repair that reads the sources alone (R1) and preserves validity (R2); it applies verbatim with Γ_j in place of ϕ_{ij} . Newman's Lemma gives (a), and the witness construction gives (b). (Coq: `fed_thm_resolved_convergence_guarded`, `fed_thm_resolved_convergence_exact`; for the unguarded system under XU, `fed_thm_resolved_convergence_corrected`; the version 1 statement is refuted by `fed_thm_resolved_convergence_refuted`.)

$\square$

$\square$

Corollary 8.25 (Constructive Resolved Normal Form). *Corollary 8.15 extends to (H_R) with C1 and C2: compute each source’s single-registry run, then, in topological order, run each target’s events with the repair against its finalized sources (its resolution operator, or authority morphism) interleaved after every event. The federated normal form is the unique solution of these equations. As in the tree case, the product state space is never materialized. (Coq: `fed_cor_resolved_nf_corrected`.)*

Remark 8.26 (Verified, not assumed). Unlike the authority morphism, a resolution operator encodes domain policy: there is no canonical merge. The theorem is therefore conditioned on (R1)–(R2), exactly as the single-source case is conditioned on M1, and on C1 and C2. All four are finite conditions on one target and its sources: on finite state spaces they are checked by enumerating $\prod_{i \in S(j)} \Sigma_{R_i} \times \Sigma_{R_j}$ (with the target’s events for C1 and C2), so a build-time verifier certifies a resolved network as it certifies WFC and CC for a single registry. Theorem 8.12 is the special case in which every target has a single source whose resolution operator is the morphism image.

Remark 8.27 (Necessity of the resolution conditions). Acyclicity remains necessary (Proposition 8.18 is unchanged; resolution operators do not tame cycles). (R2) is necessary for the reason M1 is (Proposition 8.19): a two-source resolver satisfying (R1) that maps a valid source combination to a shared component violating V_{R_j} makes resolution repair and local compensation oscillate forever, with no federally valid state (`r2_necessary`). (R1) is necessary for a unique repair normal form: a resolver that reads the target’s local component, on an acyclic network where (R2) and component WFC hold, admits two federally valid normal forms with different shared components from one state, depending on whether local compensation or resolution repair fires first (`r1_necessary`).

8.6 Monotone Cycles

Acyclicity (Definitions 8.3 and 8.22) is required because the necessity counterexamples turn on *non-monotone* repair: the cyclic counterexample (Proposition 8.18) uses the negation morphism $\phi(x) = 1 - x$, and the M1 counterexample (Proposition 8.19) an antitone flip. We show that when repair is *monotone* on an ordered shared domain, acyclicity is unnecessary for a unique repair normal form: it is a least fixed point (Knaster–Tarski), and order-independence of the repair is the classical convergence of *chaotic iteration*. Event convergence on a cycle needs a separate condition, which we also characterize exactly.

Definition 8.28 (Ordered Shared Domains). A network has *ordered shared domains* if each shared space S_j carries a partial order $\sqsubseteq_j$ making $(S_j, \sqsubseteq_j)$ a complete lattice (least element $\perp_j$; all joins exist). For fixed local components (the components’ Phase 1 normal forms), define the *federated repair operator* $\Phi : \prod_j S_j \rightarrow \prod_j S_j$ by

$$\Phi(\bar{s})_j = \begin{cases} s_j & j \text{ a source (no incoming): it keeps its own shared value,} \\ \phi_{ij}(\sigma_i[\text{sh} \mapsto s_i]) & j \text{ single-source from } i, \\ \Gamma_j((\sigma_i[\text{sh} \mapsto s_i])_{i \in S(j)}) & j \text{ multi-source.} \end{cases}$$

The morphisms and resolvers are *monotone* when Φ is monotone: $\bar{s} \sqsubseteq \bar{s}' \Rightarrow \Phi(\bar{s}) \sqsubseteq \Phi(\bar{s}')$ (componentwise). Source entries are identities, hence monotone; the condition constrains only the morphism images and resolvers. The network is *valid at bottom* if every component is locally valid with its shared component set to $\perp_j$ (for the given locals).

(Coq: Φ built from a network, `fed_def_lattice_shared_cyclic_hyps`, which shows that monotone Φ discharges the hypotheses of the cyclic normalizer of `FederationEventsCycles.v`.)

Theorem 8.29 (Monotone Convergence Despite Cycles). *Let $\mathcal{N}$ be a morphism network with ordered shared domains (Definition 8.28), possibly cyclic, whose morphisms and resolvers are monotone and validity-preserving (M1 / R2), and whose components satisfy WFC and CC.*

- (a) **Repair normal form.** Φ has a least fixed point $\bar{s}^*$. Under the ascending chain condition (ACC; in particular for a finite lattice), Kleene iteration from $\perp$ and every fair asynchronous (any-order) iteration of the component updates from $\perp$ reach $\bar{s}^*$ in finitely many steps, so the repair normal form does not depend on the schedule. Without ACC, $\bar{s}^* = \bigsqcup_{k \geq 0} \Phi^k(\perp)$ when Φ preserves the supremum of this chain (for instance, when Φ is Scott-continuous).
- (b) **Validity.** If moreover the network is valid at bottom, every Kleene iterate and every chaotic iterate from $\perp$ is locally valid, and under ACC $\bar{s}^*$ is federally valid. Without ACC the same holds when Φ preserves the supremum of the Kleene chain and local validity is closed under suprema of ascending chains.
- (c) **Events.** Let the governed step apply an event and then normalize (Phase 1, then reset the shared components to $\perp$ and iterate to $\bar{s}^*$). From a state $\bar{\sigma}^0$, all event sequences that differ by swapping federated-independent events reach the same state if and only if the global condition GC($\bar{\sigma}^0$) holds: federated-independent events commute after full re-normalization at every state reachable from $\bar{\sigma}^0$. GC holds at every normalized state when, for every target j , with H_j the set of shared values the repair writes into j from valid states (the morphism and resolver images): (**C1_{cyc}**) the local outcome of each event of j at a valid state does not depend on which value of H_j the shared component holds; and (**C2_{cyc}**) for independent events of j , the local outcomes of the two orders agree when the shared component is reset to the same value of H_j between the events.

Proof. (a) Φ is a monotone self-map of the complete lattice $\prod_j S_j$ (a product of complete lattices), so by Knaster–Tarski [14] it has a least fixed point. Every Kleene iterate lies below every fixed point, by induction from $\perp$. Under ACC the ascending chain $\perp \sqsubseteq \Phi(\perp) \sqsubseteq \dots$ stabilizes, and a stable value is a fixed point, hence $\bar{s}^*$. Order-independence is the convergence of chaotic iteration [15]: below $\bar{s}^*$ a productive monotone component update strictly increases the state, so under ACC every fair schedule from $\perp$ terminates, and its result is a fixed point below $\bar{s}^*$, hence $\bar{s}^*$. If Φ preserves the supremum s of the Kleene chain, then $\Phi(s) = \bigsqcup_k \Phi^{k+1}(\perp) = s$, and s lies below every fixed point.

(b) M1 and R2 carry local validity from a valid state to its repair, so by induction from the valid bottom every iterate is locally valid; under ACC $\bar{s}^*$ is an iterate, and as a fixed point it satisfies every morphism invariant. Without ACC, closure of validity under suprema of chains carries validity to the Kleene supremum.

(c) If GC holds, a swap of two independent events at a reachable state does not change the result, and trace-equivalent sequences are connected by such swaps, all at reachable states. Conversely, if two independent events fail to commute at a reachable state, the two one-swap sequences through that state diverge. For the per-target check: on a valid state the governed step resets every shared component to $\perp$ and recomputes it from the locals, so the event’s writes to shared components are erased and only its local outcome matters; C1_{cyc} and C2_{cyc} say that this local outcome does not depend on the order of independent events, since the shared value an event reads at a normalized state lies in H_j . □ □

(Coq: (a) `kleene_lfp`, `lfp_unique`, `kleene_acc_lfp`, `chaotic_reaches_lfp`, `chaotic_acc_reaches_lfp`, `chaotic_limit_unique`, `kleene_sup_lfp`, and for gsm’s cyclic normalizer `cyc_N_lfp`, `cyc_N_unique`, `cyc_sweep_order_independent`; (b) `net_iter_valid`, `net_sweep_valid`, `net_lfp_valid`, `net_Ncyc_correct`, `net_Ncyc_idem`, `kleene_sup_valid`,

non-vacuity `fed_valid_corrected_instance`; (c) `gc_iff`, `cyc_events_converge_iff`, `net_events_converge_iff`, `fed_thm_monotone_cycles_events_corrected`, and the per-target check `cyc_check_gc`, `cyc_check_converges`, `cyc_check_gc_lfp`, non-vacuity `cyc_check_instance`.)

Each added hypothesis is needed. Without validity at bottom the least fixed point need not be federally valid: a 2-cycle of identity morphisms on $\{false \sqsubset true\}$, valid iff the flag is set, meets every other hypothesis, yet its least fixed point $(false, false)$ is invalid while $(true, true)$ is a valid fixed point (`fed_thm_monotone_cycles_lfp_invalid`). Without continuity the Kleene supremum need not be a fixed point: on the complete chain $0 < 1 < \dots < \omega < \omega+1$ with $\Phi(k) = k+1$ and $\Phi(\omega) = \Phi(\omega+1) = \omega+1$, the supremum ω is not fixed (`fed_thm_monotone_cycles_kleene_formula_fails`). Without GC, events diverge even when every hypothesis of (a) and (b) holds: an event that copies a shared flag fed around a monotone 2-cycle into a local alarm ends in different states under the two orders of one allowed swap (`cyc_counterexample`, `fed_thm_monotone_cycles_events_refuted`). That event fails $C1_{cyc}$ for every choice of H_j (`check_rejects_latch`); $C2_{cyc}$ cannot be dropped either (`c1_localcc_insufficient`), nor replaced by monotonicity of the events (`monotone_c2_insufficient`). On a cycle the repair equations can also have fixed points above the least one (`bottom_matters`), which is why the normalizer resets to $\perp$.

Corollary 8.30 (Acyclicity is a concession to non-monotonicity). *For the repair normal form, acyclicity is needed only to tame non-monotone repair. The cyclic divergence of Proposition 8.18 uses the antitone negation $\phi(x) = 1 - x$, which is not monotone (`negation_not_monotone`); under monotonicity Theorem 8.29(a)–(b) applies. Acyclic structure and monotonicity are thus two independent routes to a unique federated repair normal form, the former for arbitrary (possibly non-monotone) repair, the latter for arbitrary (possibly cyclic) topology. Neither gives event convergence by itself: that needs $C1$ and $C2$ on acyclic networks (Theorems 8.12 and 8.24) and GC on monotone cycles (Theorem 8.29(c)).*

Remark 8.31 (Non-monotone cycles under coordination). A non-monotone cycle can still converge if some of its edges are coordinated. In the invertible fragment (each shared fiber a group acting on itself, each edge a transport by a group element), pick an authority root and a spanning tree; drive values along the tree, keep the non-tree edges whose loop is balanced as checked constraints, and coordinate the unbalanced ones outside the federation. The result is the unique state satisfying the tree and the balanced edges with the root’s value, independent of the topological order (`coordinated_sound`, `coordinated_unique_nf`); permutations of local events converge when the root’s own events commute (`coordinated_events_converge`); and the coordinated set is exactly the unbalanced edges, no fewer and no more (`plan_exact`, `coordination_needed`).

Remark 8.32 (Order-independence is chaotic iteration). Theorem 8.29(a) exposes the central property of normalization confluence in another guise: convergence of the repair “regardless of application order” is precisely the order-independent convergence of chaotic iteration in dataflow analysis and abstract interpretation [15]. The monotone regime inherits that machinery (worklist scheduling) for computing the federated repair normal form; see Remark 8.34 for what widening does and does not give.

Remark 8.33 (CRDTs are compensation-free monotone federations). A join-semilattice whose merge is its join is monotone and trivially validity-preserving, and with every state valid the network is valid at bottom, so a state-based CRDT [3] fits Theorem 8.29(a)–(b) as the case in which repair never has to restore an invariant. (Coq proves CRDT convergence directly, `cvrtd_SEC`, not as an instance of the theorem.) Normalization confluence adds compensation on top: monotone convergence *with* business invariants, which operation commutativity alone does not provide.

Remark 8.34 (Infinite lattices and widening). Theorem 8.29 admits a *complete* lattice, which may be infinite. Existence and uniqueness of the least fixed point $\bar{s}^*$ hold on any complete lattice (Knaster–Tarski). Finite iteration reaches it under ACC. Without ACC (an unbounded counter or resource in $\mathbb{N} \cup \{\top\}$, whose ascending chains never stabilize), iteration may never reach it, even for continuous Φ : on $0 < 1 < \dots < \omega$ with $\Phi(k) = k+1$ and $\Phi(\omega) = \omega$, the least fixed point ω is reached by no iterate (`kleene_sup_instance`); and for non-continuous Φ not even the limit is the least fixed point (Theorem 8.29 and the counterexample after it). A *widening* operator (abstract interpretation [15]) forces termination by extrapolating to a post-fixed point w , $\Phi(w) \sqsubseteq w$, which bounds every iterate and the least fixed point (`widening_sound`). In general w is not a fixed point, so it does not satisfy the morphism invariants and is not the federated normal form (`widening_not_normal_form`). Widening gives a sound bound, not convergence.

8.7 Compositionality: Sub-Federations as Effective Registries

Product composition (Theorem 10.15) collapses *independent* registries into one. The same holds for a *connected* sub-federation: an internally convergent sub-network can be replaced by a single *effective registry*, so federations verify modularly and hierarchically. Registries and morphisms form a category in which the federated registry is the limit of the constraint diagram; this subsection is the statement that such limits compose.

Definition 8.35 (Sub-Federation, Boundary, Convexity). Given $\mathcal{N} = (\{R_i\}_{i \in \mathcal{I}}, \{\phi_{ij}\})$ and $\mathcal{J} \subseteq \mathcal{I}$, the *sub-federation* on $\mathcal{J}$ has components $\{R_i\}_{i \in \mathcal{J}}$ and *internal* morphisms $E_{\mathcal{J}} = \{(i, j) \in E : i, j \in \mathcal{J}\}$. Its boundary consists of *import* edges (from $\mathcal{I} \setminus \mathcal{J}$ into $\mathcal{J}$) and *export* edges (from $\mathcal{J}$ out). $\mathcal{J}$ is *convex* if no directed path between two members of $\mathcal{J}$ passes through a vertex outside $\mathcal{J}$.

Definition 8.36 (Effective Registry). The *effective registry* $R_{\mathcal{J}}$ is the federated registry (Definition 8.4) of the sub-federation: state space $\prod_{i \in \mathcal{J}} \Sigma_{R_i}$, invariants the internal local and morphism invariants, and compensation the sub-federated normalizer $\rho_{\text{Fed}}^{\mathcal{J}}$. Its shared component is the union of the import ports (the shared components of members targeted by import edges); export edges read its normalized state.

Theorem 8.37 (Compositional Collapse). *Let $\mathcal{N}$ be acyclic with M1/R2 on its morphisms and resolvers, R1 on its resolvers, and WFC and CC on its components, and let $\mathcal{J}$ be convex. Write $C1(\mathcal{J})$, $C2(\mathcal{J})$ for $C1$ and $C2$ (Definition 8.7) restricted to events of members of $\mathcal{J}$. Then:*

- (a) **WFC.** *For every state whose non-members are valid, $\rho_{\text{Fed}}^{\mathcal{J}}$ returns an $R_{\mathcal{J}}$ -valid state (every member locally valid, every internal morphism invariant satisfied) and changes no non-member; it is the identity on $R_{\mathcal{J}}$ -valid states. So $R_{\mathcal{J}}$ satisfies WFC with $\rho_{R_{\mathcal{J}}}^* = \rho_{\text{Fed}}^{\mathcal{J}}$.*
- (a') **CC.** *If $C1(\mathcal{J})$ and $C2(\mathcal{J})$ hold, then at every $R_{\mathcal{J}}$ -valid state whose non-members are valid, any two federated-independent events of members of $\mathcal{J}$ commute through the governed step $\rho_{\text{Fed}}^{\mathcal{J}} \circ \text{apply}(e, -)$ of $R_{\mathcal{J}}$. Exactly: the event system of $R_{\mathcal{J}}$ converges from an $R_{\mathcal{J}}$ -valid state iff $C1$ and $C2$ hold at the witnesses reachable inside $\mathcal{J}$.*
- (b) **Boundary.** *Read $R_{\mathcal{J}}$'s validity, for the purpose of M1, as the product of its members' validities. Then an import morphism into a member j satisfies M1 into $R_{\mathcal{J}}$ iff it satisfies M1 into R_j , and an export morphism satisfies M1 read from $R_{\mathcal{J}}$ iff it does in $\mathcal{N}$. Moreover, after any import (a non-member takes a new valid value and the morphism into a member is applied), $\rho_{\text{Fed}}^{\mathcal{J}}$ restores the federal validity of $R_{\mathcal{J}}$ without changing any non-member.*
- (c) **Contraction.** *Some topological order of $\mathcal{N}$ lists $\mathcal{J}$ as one contiguous block. The contraction $\mathcal{N}'$ (with $\mathcal{J}$ replaced by $R_{\mathcal{J}}$) is acyclic, and ρ_{Fed} on $\mathcal{N}'$ equals ρ_{Fed} on $\mathcal{N}$ under $\Sigma_{R_{\mathcal{J}}} = \prod_{i \in \mathcal{J}} \Sigma_{R_i}$; it factors as Phase 1, the registries before $\mathcal{J}$, $\rho_{\text{Fed}}^{\mathcal{J}}$, and the registries*

after $\mathcal{J}$. Every event run of $\mathcal{N}'$ equals the same run of $\mathcal{N}$, so $\mathcal{N}'$ converges iff $\mathcal{N}$ does (for the guarded and for the unguarded system), their normal forms agree, and both converge from every federally valid state if $C1$ and $C2$ hold; exactly, from $\bar{\sigma}^0$ iff $C1$ and $C2$ hold at the witnesses reachable from $\bar{\sigma}^0$.

Proof. (a) $\rho_{\text{Fed}}^{\mathcal{J}}$ is Phase 1 on the members followed by the internal repairs in topological order; Lemma 8.23 applied inside $\mathcal{J}$, with the import sources held fixed, gives validity, and the repairs touch only members. (a') Theorem 8.11 and Theorem 8.12(b) applied to the sub-federation. (b) An import into j overwrites only j 's shared component, so its M1 obligation is the one into R_j ; after it, the internal repairs re-establish the internal invariants by (a). (c) Order the non-members that no path from $\mathcal{J}$ reaches, then $\mathcal{J}$, then the rest; convexity makes this a topological order, and it makes the contraction acyclic. With $\mathcal{J}$ contiguous, ρ_{Fed} on $\mathcal{N}$ is literally ρ_{Fed} on $\mathcal{N}'$ with $R_{\mathcal{J}}$'s compensation inlined (the fold-append law of the staged normalizer), and an event of a member changes one member and is followed by the same repair on both sides. $\square$ $\square$

(Coq: (a) `collapse_a_wfc`; (a') `collapse_a_guarded`, `collapse_a_guarded_perm`, `collapse_a_guarded_exact`; (b) `collapse_b_import_iff`, `collapse_b_export_iff`, `collapse_b_import_corrected`, `collapse_b_export_corrected`; (c) `collapse_block_order`, `collapse_contract_topo`, `collapse_contract_acyclic`, `collapse_nf_agree`, `collapse_nf_factor`, `collapse_c_runs_agree`, `collapse_c_traceconv`, `collapse_c_guarded_iff`, `collapse_c_unguarded_iff`, `collapse_c_guarded_exact`, `collapse_c_guarded_c1_c2`; fold-append law `rhoFold_compositional`; non-vacuity `chain_collapse_instance`.)

Each qualifier is needed. Component CC does not give (a'): the federation of Remark 8.8, taken whole as $\mathcal{J}$, has two events that do not commute through $R_{\mathcal{J}}$ (`collapse_a_refuted`). Not every topological order lists a convex $\mathcal{J}$ contiguously: three independent registries with $\mathcal{J} = \{0, 2\}$ and the order 0, 1, 2 (`collapse_every_order_refuted`). Without convexity the contraction can be cyclic: $0 \rightarrow 1 \rightarrow 2$ with $\mathcal{J} = \{0, 2\}$ (`collapse_convexity_needed`). If $R_{\mathcal{J}}$'s validity includes the internal morphism invariants (Definition 8.36), the M1 equivalences of (b) fail in both directions: a new import value can break an internal copy invariant, and an export that is valid only on consistent states satisfies M1 over $R_{\mathcal{J}}$ -valid sources but not over all locally valid ones (`collapse_b_import_refuted`, `collapse_b_export_refuted`); the restoration statement of (b) is what holds in that reading. The case of a cyclic $\mathcal{J}$ with monotone repair is not covered by the mechanization; on paper, (a), (a') and (c) carry over with Theorem 8.29 in place of Lemma 8.23 and Theorem 8.12 (validity at bottom, ACC, and $C1_{\text{cyc}}$, $C2_{\text{cyc}}$ inside $\mathcal{J}$).

Corollary 8.38 (Modular and Hierarchical Verification). *$C1$ for $\mathcal{N}$ is $C1(\mathcal{J})$ together with $C1(\mathcal{I} \setminus \mathcal{J})$, and likewise for $C2$. So a federation verifies compositionally: certify each convex sub-federation once (its components, M1/R1/R2, and $C1$ and $C2$ for its members' events), replace it by its effective registry, and certify the rest; this gives the guarded CC of each $R_{\mathcal{J}}$ and the convergence of $\mathcal{N}'$ and of $\mathcal{N}$. The construction nests: regrouping consecutive nodes of a topological order into one node does not change the normalizer, and any block decomposition whose flattening is a topological order of $\mathcal{N}$ computes ρ_{Fed} , which gives hierarchical verification and black-box reuse of a certified sub-federation with a fixed port interface. Theorem 10.15 is the special case $E_{\mathcal{J}} = \emptyset$ with empty boundary; a $\mathcal{J}$ with no edge leaving it is convex.*

(Coq: `collapse_modular_c1`, `collapse_modular_c2`, `collapse_modular_converges`, `collapse_regroup`, `collapse_hierarchical`, `collapse_product_convex`.)

Remark 8.39 (Convexity and the monotone regime). Convexity is used to keep the collapsed network acyclic. In the monotone regime a boundary-induced cycle is harmless provided $\rho_{\text{Fed}}^{\mathcal{J}}$ is monotone, and then Theorem 8.29 applies to $\mathcal{N}'$ (with validity at bottom for Theorem 8.29(b))

and GC for events). Monotone repair inside $\mathcal{J}$ is *not* enough for this: $\rho_{\text{Fed}}^{\mathcal{J}}$ begins with the components' own normalizers (Phase 1), which need not be monotone. A one-registry sub-federation with no internal edges has constant, hence monotone, repair, yet its normalizer sends $a_0 \sqsubseteq a_1$ to $a_2 \not\sqsubseteq a_1$ (`fed_rem_convexity_refuted`). If Phase 1 is monotone and Φ is monotone in the locals as well as in the shared values, then under ACC $\rho_{\text{Fed}}^{\mathcal{J}}$ is monotone (`fed_rem_convexity_corrected`, `lfp_mono_param`; non-vacuity `convexity_corrected_instance`).

8.8 Example: Manufacturer–Supplier Federation

Manufacturer R_M : $\Sigma_M = \{\text{draft}, \text{active}, \text{suspended}\}$, all locally valid. e_{pub} : `draft` $\rightarrow$ `active`.

Supplier R_S : $\Sigma_S = \{\text{idle}, \text{listed}, \text{stale}, \text{err}\}$; $V_S(\text{err}) = \perp$, rest valid; $\rho_S(\text{err}) = \text{idle}$. e_{exp} : sets state to `stale`.

Total morphism ϕ : `draft` $\mapsto$ `idle`, `active` $\mapsto$ `listed`, `suspended` $\mapsto$ `stale`. M1 holds (all images are valid).

Convergence trace from `(draft, idle)` with independent events e_{pub} (targets R_M) and e_{exp} (targets R_S):

Order 1 (e_{pub} first): `(active, idle)` $\xrightarrow{\phi}$ `(active, listed)` $\xrightarrow{e_{\text{exp}}}$ `(active, stale)` $\xrightarrow{\phi}$ `(active, listed)`.

Order 2 (e_{exp} first): `(draft, stale)` $\xrightarrow{\phi}$ `(draft, idle)` $\xrightarrow{e_{\text{pub}}}$ `(active, idle)` $\xrightarrow{\phi}$ `(active, listed)`.

Both converge to `(active, listed)`. The supplier event's effect is overwritten by morphism repair: the manufacturer is authoritative. This is an instance of Theorem 8.12, not a coincidence of the two traces: M1, the supplier's WFC, C1 and C2 all hold (the supplier's event writes only state that the final repair overwrites, so both sides of C1 agree), and every reordering of any event list converges from every federally valid state (Coq: `ms_instance`, `ms_paper_traces`, `ms_converges`).

Remark 8.40 (Machine-checked). The federated results of this section are mechanized in Coq/Rocq, in the `coq/` directory of the artifact, against the paper's own hypotheses: `FederationGRS.v` states (H) and (H_R) as a record (shared/local decomposition, acyclic network, M1, R1, R2, component WFC with ρ^* as the iterate, component CC1 and CC2) and proves every result above under its label, for the guarded system $\mathcal{G}_{\text{Fed}}$ and for the unguarded one. The machine model of a federated processor and the sufficiency of C1 and C2 are in `FederationEvents.v`; the exact converses in `FederationEventsConverse.v`; monotone cycles in `FederationEventsCycles.v`, `FederationEventsCyclesCheck.v`, `MonotoneFederation.v`, `Chaotic.v` and `ChaoticACC.v`; the compositional collapse in `Collapse.v`; coordinated cycles in `CoordinatedCycles.v`; and the concrete examples (Propositions 8.18 and 8.19, the resolution conditions, and the manufacturer and supplier) in `PaperInstances.v`. Each refuted statement of version 1 has a gated counterexample against its exact hypotheses (Appendix A). The theorems named in this paper are in the `Print Assumptions` gate of `coq/verify.sh` (669 theorems on the main branch at the time of writing, 715 with `Collapse.v`), and each is “Closed under the global context”: no axioms, no admitted lemmas. Not mechanized: the cyclic monotone case of Theorem 8.37, and the decidability remark for R1, R2, C1 and C2.

9 Discussion

9.1 Three Regimes of Coordination-Free Convergence

Our result, combined with CRDTs [3] and invariant confluence [6], identifies three distinct structural regimes under which distributed processors converge without coordination:

1. **Operation commutativity** (CRDTs): Operations commute algebraically. Convergence follows from the algebra of operations. No invariant enforcement or compensation is needed.
2. **Invariant confluence** (I-confluence): Operations preserve application invariants under all orderings. Convergence follows because validity is never violated. No compensation is needed.
3. **Normalization confluence** [1]: Operations may be non-commutative and may violate invariants. Compensation normalizes states to validity. WFC and CC are sufficient for semantic convergence (Theorem 5.5); UBC is orthogonal to convergence and governs scalability (Theorem 7.1). No coordination is needed.

The regimes relate as follows. Operation commutativity with every state valid implies CC (with $\rho_R^* = \text{id}$), and conversely a compensation-free system meets the causal convergence condition exactly when it is an op-based CRDT, so normalization confluence strictly contains the op-based CRDTs. Invariant confluence implies that compensation never fires, but it does not by itself make operations commute: in the sequential-application model of this paper, an invariant-confluent system satisfies CC exactly when its operations commute on the reachable states, and by Theorem 6.5 one whose operations do not commute there diverges under free delivery. So normalization confluence contains the invariant-confluent systems that converge under reordering, not all of them, and it additionally covers systems where operations are non-commutative and invariant-violating, the common case in business applications.

The regimes partition the coordination-free design space by *where* convergence is guaranteed: in the operations (regime 1), in the invariants (regime 2), or in the normalization rewrite system (regime 3).

Remark 9.1 (Convergence as Rewrite Property). The conventional literature frames convergence as a property of operations: operations must commute, or operations must preserve invariants. Normalization confluence reframes convergence as a property of the *rewrite system* that interleaves application and compensation. The operations themselves need no special algebraic structure. This shifts the design burden from operation algebra to compensation design, and places the convergence mechanism at the semantic normalization layer—above the data-structure layer (CRDTs) and the transaction layer (I-confluence). Moreover, this rewrite-system perspective extends across organizational boundaries: in the federated results (Section 8) multi-registry systems inherit termination and a unique repair normal form from their components, with morphism repair as additional rewrite rules in the same framework; event convergence additionally needs two per-edge checks, C1 and C2.

9.2 Two Regimes for Federated Repair

The federated results partition constraint-network design into two regimes with *independent* routes to a unique repair normal form (Corollary 8.30). When repair is *monotone* on ordered shared domains, the repair normal form exists on *any* topology, including cyclic meshes, by Theorem 8.29 (federally valid when the network is valid at bottom). When repair may be *non-monotone* (toggles, cancellations, order-reversing revocations), the topology must be *acyclic*, with resolution operators (Section 8.5) at multi-source targets, and the normal form is the

one-shot constructive one (Corollary 8.25), unless the offending cycles are coordinated (Remark 8.31). In both regimes repair must preserve validity (M1/R2), and in both, convergence of *event interleavings* needs a further check: C1 and C2 per edge on acyclic networks, GC on monotone cycles (implied by per-target C1 and C2 over the repair images). Table 1 summarizes the trade.

Regime	Repair requirement	Topologies	Repair form	normal	Event convergence
Monotone	Monotone on a lattice order (join or meet); validity-preserving; valid at bottom	Any: cyclic meshes, DAGs, trees	Least fixed point $\text{lfp}(\perp)$ under any <i>fair</i> asynchronous order (Knaster–Tarski, chaotic iteration, ACC)		Iff GC; implied by per-target C1, C2 over images
Non-monotone	May reverse the order (toggles, negation, cancellations); validity-preserving	Acyclic only; multi-source targets need resolution operators	Deterministic one-shot constructive normal form		Under C1, C2; exact at reachable witnesses

Table 1: Two regimes for federated repair. Monotonicity and acyclicity are independent conditions for a unique repair normal form: monotone repair on any topology (Theorem 8.29); non-monotone repair requires acyclicity (Proposition 8.18). Validity preservation (M1/R2) is required in both, and event convergence needs the checks in the last column (Theorems 8.12, 8.24 and 8.29).

9.3 Verifying Compensation Commutativity

CC is easy to state but potentially hard to verify: it quantifies over all states and all independent event pairs. Section 10 develops a verification calculus that reduces CC to local per-event and per-pair checks via invariant footprints and decomposable repair. Here we note three common patterns that practitioners can use as a first check before invoking the full calculus:

1. **Compensation to a canonical state.** Suppose there is a default valid state $\sigma_\perp$ such that $\rho_R^*(\sigma) = \sigma_\perp$ for all invalid σ (the “reset to safe default” pattern). This alone does not give CC: on valid states compensation never fires, so two valid-to-valid events that do not commute violate CC1, and an event that treats an invalid state differently from the reset state violates CC2. What the pattern does give is a cheap check. CC2 holds for e if and only if $N(A_e(\sigma)) = N(A_e(\sigma_\perp))$ for every invalid σ (one comparison per invalid state against a fixed reference), where $N = \rho_R^*$ and $A_e = \text{apply}(e, -)$ as in Section 10; and when this holds for e_1 and e_2 , CC1 for the pair at σ holds if and only if $N(A_{e_2}(A_{e_1}(\sigma))) = N(A_{e_1}(A_{e_2}(\sigma)))$. So canonical compensation, the reset check, and commutation up to N together give CC1 and CC2.
2. **Lattice-structured compensation.** Suppose the states form a join-semilattice, each event joins a fixed element ($\text{apply}(e, \sigma) = \sigma \sqcup a_e$), the normalizer N is a closure operator (monotone, extensive, idempotent), and one compensation step stays between a state and its closure ($\sigma \sqsubseteq \rho_R(\sigma) \sqsubseteq N(\sigma)$). Then $N(N(x) \sqcup y) = N(x \sqcup y)$ for all x, y , so both sides of CC1 equal $N(\sigma \sqcup a_{e_1} \sqcup a_{e_2})$, and CC2 holds because $\sigma \sqcup a_e \sqsubseteq \rho_R(\sigma) \sqcup a_e \sqsubseteq N(\sigma) \sqcup a_e$. (This is a short paper-level argument; it is not part of the mechanization.) Monotone repair on an ordered shared domain is also the setting of the cyclic federations of Section 8.6, where federated repair reaches a least fixed point (by the Knaster–Tarski theorem) and a further condition

decides whether event order matters, with state-based CRDTs as the compensation-free special case.

3. **Decomposable repair.** If compensation decomposes into commuting per-invariant repairs, CC can be verified via the footprint calculus of Section 10. The order fulfillment example (Section 4) uses this pattern, with one invariant whose repair is the hold.

9.4 Applications

The compensate-and-converge principle applies wherever systems process streams of changes under semantic constraints with bounded compensation. We highlight three domains.

Infrastructure Configuration Drift. Infrastructure-as-code tools apply streams of configuration changes to infrastructure state. Valid configurations are defined by policy. The semantic state space is finite (k boolean policy predicates yield at most 2^k equivalence classes). Remediation operators that reset violated policies to compliant defaults fit the canonical-state pattern: CC2 reduces to one comparison per violating state against the default, and CC1 to commutation of changes up to normalization. When those checks pass, our theorem guarantees convergence to equivalent policy-compliant states regardless of the order in which changes are applied.

Database Schema Migration. Schema migrations are event streams applied to schema state. The set of valid schema configurations is finite. Migration scripts that repair constraint violations constitute compensation operators. When such scripts resolve each constraint independently (decomposable repair), CC reduces to the per-event and per-pair checks of the calculus (Section 10); when those pass, our theorem guarantees order-independent convergence.

Regulatory Compliance. Organizations processing streams of regulatory changes must maintain compliance. Compliance states are finite (each regulation is either satisfied or not). Remediation procedures that restore compliance by addressing each violated regulation independently are decomposable repairs, so CC can be checked with the calculus; when it holds, our theorem guarantees convergence to equivalent compliance states.

Cross-Organizational Supply Chains. The federated convergence result (Section 8) applies to multi-registry settings. A manufacturer’s product specification registry connected by morphisms to supplier capability registries forms a tree-shaped network. Each organization maintains its own compensation operators; the morphisms encode specification conformance. When each supplier’s events pass C1 and C2 against the manufacturer’s images (a per-edge check), Theorem 8.12 guarantees that all participants converge to consistent states across organizational boundaries. When a supplier answers to *several* authorities at once—say a manufacturer’s specifications *and* a regulator’s rules—the network is no longer a tree; a resolution operator (Section 8.5) combines the constraints (for instance, most-restrictive-wins), and Theorem 8.24 extends the same guarantee to this acyclic multi-source setting.

9.5 Extensions

At-least-once delivery. The convergence theorems assume each event is delivered once. Real transports deliver at least once, so an event may be redelivered, possibly much later. A duplicate is absorbed when the redelivered event’s governed step is idempotent and commutes

with every event delivered between its copies (Coq: `alo_absorbed`); if all events commute and every duplicated event is idempotent, any two at-least-once deliveries of the same events agree (`alo_commuting_converges`); under causal delivery only concurrent events need to commute, provided redelivery is itself causally consistent (`causal_alo_exactly_once`, `causal_alo_converges`). Both qualifiers are needed: a duplicate of an event whose governed step is not idempotent diverges (`non_idempotent_diverges`), and a late redelivery of an idempotent event after its causal successor can diverge (`late_duplicate_diverges`).

Privacy. One extension remains future work: *privacy-preserving validation* where organizations prove registry compliance via zero-knowledge proofs without revealing state. The federated convergence result (Section 8) handles the multi-registry coordination problem; privacy-preserving validation would additionally protect organizational sovereignty over state visibility.

10 Verification Calculus for CC

Compensation commutativity is a global condition: it quantifies over all states and all independent event pairs. This section develops conditions, exact where possible, that reduce CC to local per-event and per-pair checks. The key tools are an *invariant footprint* mapping and a *decomposable repair* condition.

10.1 Notation

For readability in algebraic manipulations, we write $N(\sigma) := \rho_R^*(\sigma)$ for iterated compensation to validity (Definition 3.15). The normalizer satisfies: (i) $V_R(N(\sigma)) = \top$ for all σ ; (ii) $N(\sigma) = \sigma$ if $V_R(\sigma) = \top$; (iii) $N(N(\sigma)) = N(\sigma)$ (idempotence) (`rho_star_valid`, `rho_star_fix`, `rho_star_idem`). We write $A_e(\sigma) := \text{apply}(e, \sigma)$ when convenient.

In this notation, the CC obligations become:

- (CC1) $N(A_{e_2}(N(A_{e_1}(\sigma)))) = N(A_{e_1}(N(A_{e_2}(\sigma))))$ for independent e_1, e_2 .
- (CC2) $N(A_e(\sigma)) = N(A_e(\rho_R(\sigma)))$ for invalid σ .

We say that CC1 holds for (e_1, e_2) at σ when its equation holds at that state, and that CC2 holds for e when its equation holds at every invalid σ .

10.2 Invariant Vectors and Footprints

Definition 10.1 (Invariant Vector). Define $\mathbf{v} : \Sigma_R \rightarrow \{0, 1\}^k$ by $\mathbf{v}(\sigma)_i = 1 \iff \psi_i(\sigma) = \top$.

Definition 10.2 (Footprint). A *footprint* is a function $F : \mathcal{E} \rightarrow 2^{\{1, \dots, k\}}$ such that for all σ and all $i \notin F(e)$,

$$\mathbf{v}(A_e(\sigma))_i = \mathbf{v}(\sigma)_i.$$

Intuitively, $F(e)$ is the set of invariants whose truth value can change under e . Over-approximations are sound: larger footprints make the calculus more conservative but never incorrect.

10.3 Decomposable Repair

We capture the common case where repair addresses each violated invariant independently.

Definition 10.3 (Per-Invariant Repair). A set of *per-invariant repair operators* $r_1, \dots, r_k : \Sigma_R \rightarrow \Sigma_R$ satisfies:

- (R1) **Targeted fix:** if $\psi_i(\sigma) = \perp$, then $\psi_i(r_i(\sigma)) = \top$.
- (R2) **No-op on satisfied:** if $\psi_i(\sigma) = \top$, then $r_i(\sigma) = \sigma$.
- (R3) **Mutual commutativity:** $r_i(r_j(\sigma)) = r_j(r_i(\sigma))$ for all i, j .

Definition 10.4 (Decomposable Normalizer). The normalizer N is *decomposable* if there exist per-invariant repairs $r_1, \dots, r_k$ as above such that

$$N(\sigma) = r_1(r_2(\dots r_k(\sigma) \dots))$$

for all σ . (The order is irrelevant by mutual commutativity.)

Definition 10.5 (Repair Locality). An event e is *repair-local* with respect to footprint $F(e)$ if for all $i \notin F(e)$,

$$r_i(A_e(\sigma)) = A_e(r_i(\sigma)).$$

That is, e commutes with repairs of invariants it does not affect.

10.4 Strong Absorption and CC2

Theorem 10.6 (Strong Absorption). Assume WFC and let $N = \rho_R^*$. Then CC2 holds (for all events) if and only if, for all events e and states σ ,

$$N(A_e(\sigma)) = N(A_e(N(\sigma))).$$

Proof. ($\Leftarrow$) Let $V_R(\sigma) = \perp$. Because N iterates ρ_R to the unique valid fixpoint reachable from σ , and $\rho_R(\sigma)$ lies on the same compensation chain, we have $N(\sigma) = N(\rho_R(\sigma))$. Applying the assumed equation at σ and at $\rho_R(\sigma)$:

$$N(A_e(\sigma)) = N(A_e(N(\sigma))) = N(A_e(N(\rho_R(\sigma)))) = N(A_e(\rho_R(\sigma))),$$

where the first and third equalities use the assumption and the middle equality uses $N(\sigma) = N(\rho_R(\sigma))$. This is CC2.

($\Rightarrow$) This is the generalization of CC2 noted in Remark 3.18: by induction on the number m of compensation steps from σ to $N(\sigma) = \rho_R^m(\sigma)$, CC2 at each invalid $\rho_R^j(\sigma)$, $j < m$, gives $N(A_e(\rho_R^j(\sigma))) = N(A_e(\rho_R^{j+1}(\sigma)))$, and chaining these equalities gives $N(A_e(\sigma)) = N(A_e(N(\sigma)))$. $\square$

Both directions are mechanized (`strong_absorption_iff_cc2`; the forward direction alone is `base_thm_strong_absorption`). The per-event form also holds: CC2 for a single event e at every invalid state implies strong absorption for e (`calc_cc2_strong_absorption`).

Remark 10.7. Strong absorption states: “normalizing before applying an event does not change the final normalized outcome.” This holds whenever compensation repairs violations in a way determined by the violation pattern rather than by the path to the violation. The order fulfillment example (Section 4) satisfies this: the held state captures the approval intent regardless of whether the order was directly approved with low balance or arrived at the invalid state through a different path.

10.5 Footprints and CC1

We prove two preparatory lemmas before the main theorem.

Lemma 10.8 (Repair-Event Commutation). *If event e is repair-local with respect to footprint $F(e)$ (Definition 10.5), then for any index set J with $J \cap F(e) = \emptyset$ and any state σ :*

$$R_J(A_e(\sigma)) = A_e(R_J(\sigma)),$$

where $R_J = \prod_{i \in J} r_i$.

Proof. By repair locality, $r_i \circ A_e = A_e \circ r_i$ for each $i \in J$ (since $i \notin F(e)$). The result follows by induction on $|J|$: if the claim holds for $R_{J'}$ with $|J'| < |J|$, then for any $j \in J$,

$$R_J(A_e(\sigma)) = r_j(R_{J \setminus \{j\}}(A_e(\sigma))) = r_j(A_e(R_{J \setminus \{j\}}(\sigma))) = A_e(r_j(R_{J \setminus \{j\}}(\sigma))) = A_e(R_J(\sigma)),$$

using the inductive hypothesis and then $r_j \circ A_e = A_e \circ r_j$. $\square$

Lemma 10.9 (Repair Block Idempotence). *For any index set $J \subseteq \{1, \dots, k\}$, the repair block R_J is idempotent on states that are valid on all invariants in J : if $\psi_i(\sigma) = \top$ for all $i \in J$, then $R_J(\sigma) = \sigma$. More generally, $R_J(R_J(\sigma)) = R_J(\sigma)$ for all σ .*

Proof. If $\psi_i(\sigma) = \top$ for all $i \in J$, then each $r_i(\sigma) = \sigma$ by (R2), so $R_J(\sigma) = \sigma$. For general idempotence: $R_J(\sigma)$ satisfies $\psi_i(R_J(\sigma)) = \top$ for all $i \in J$ (each r_i fixes its invariant by (R1) and does not break others by mutual commutativity and (R2)), so $R_J(R_J(\sigma)) = R_J(\sigma)$ by the first claim. $\square$

Theorem 10.10 (Footprint Commutativity). *Let $N = \rho_R^*$ be decomposable via per-invariant repairs $r_1, \dots, r_k$ satisfying (R1) to (R3), let F be a footprint, and let e_1, e_2 be events. Write $F_j = F(e_j)$ and consider the hypotheses*

- (H1) *Disjoint footprints: $F_1 \cap F_2 = \emptyset$.*
- (H2) *Repair locality: each e_j is repair-local with respect to F_j .*
- (H3) *Commutation up to N at σ : $N(A_{e_2}(A_{e_1}(\sigma))) = N(A_{e_1}(A_{e_2}(\sigma)))$ (raw commutation $A_{e_1} \circ A_{e_2} = A_{e_2} \circ A_{e_1}$ suffices).*
- (H4) *Footprint absorption: $N(A_{e_j}(R_{F_j}(\tau))) = N(A_{e_j}(\tau))$ for all states τ and $j = 1, 2$.*

Then:

- (a) **Exact form.** *Under (H1) and (H2), for every state σ , CC1 holds at σ if and only if $N(A_{e_2}(A_{e_1}(R_{F_2}(\sigma)))) = N(A_{e_1}(A_{e_2}(R_{F_1}(\sigma))))$.*
- (b) **Valid states.** *Under (H1) and (H2), for every valid σ , CC1 holds at σ if and only if (H3) holds at σ . In particular, (H1), (H2) and raw commutation give CC1 at every valid state.*
- (c) **Every state.** *Under (H2), (H3) at σ and (H4), CC1 holds at σ ; (H1) is not needed. Given (H2), footprint absorption (H4) for e_j is equivalent to strong absorption for e_j : $N(A_{e_j}(\tau)) = N(A_{e_j}(N(\tau)))$ for all τ .*

Proof. Write J^c for $\{1, \dots, k\} \setminus J$. First, $N \circ R_J = N$ for every index set J : by decomposability and (R3), $N = R_{J^c} \circ R_J$, so $N(R_J(\tau)) = R_{J^c}(R_J(R_J(\tau))) = R_{J^c}(R_J(\tau)) = N(\tau)$ by Lemma 10.9. Second, if e is repair-local with respect to $F(e)$, then for every τ

$$N(A_e(N(\tau))) = N(A_e(R_{F(e)}(\tau))), \tag{1}$$

because $N(\tau) = R_{F(e)^c}(R_{F(e)}(\tau))$, Lemma 10.8 moves $R_{F(e)^c}$ past A_e , and $N \circ R_{F(e)^c} = N$.

(a) By (1) for e_2 , the left side of CC1 is $N(A_{e_2}(N(A_{e_1}(\sigma)))) = N(A_{e_2}(R_{F_2}(A_{e_1}(\sigma))))$, and by (H1) and Lemma 10.8 for e_1 (with $J = F_2$, disjoint from F_1) this is $N(A_{e_2}(A_{e_1}(R_{F_2}(\sigma))))$. Symmetrically, the right side is $N(A_{e_1}(A_{e_2}(R_{F_1}(\sigma))))$.

(b) At a valid σ every $\psi_i(\sigma)$ holds, so $R_{F_1}(\sigma) = R_{F_2}(\sigma) = \sigma$ by Lemma 10.9, and (a) becomes (H3).

(c) By (1), (H4) for e_j says exactly $N(A_{e_j}(N(\tau))) = N(A_{e_j}(\tau))$ for all τ , which is strong absorption (conversely, strong absorption at $R_{F_j}(\tau)$ together with $N(R_{F_j}(\tau)) = N(\tau)$ gives (H4)). Then

$$N(A_{e_2}(N(A_{e_1}(\sigma)))) = N(A_{e_2}(A_{e_1}(\sigma))) = N(A_{e_1}(A_{e_2}(\sigma))) = N(A_{e_1}(N(A_{e_2}(\sigma)))),$$

by strong absorption for e_2 , (H3), and strong absorption for e_1 . $\square$

Corollary 10.11 (CC1 Under CC2). *Assume WFC and let $N = \rho_R^*$. If CC2 holds for e_1 and e_2 , then for every state σ , CC1 holds for the pair at σ if and only if $N(A_{e_2}(A_{e_1}(\sigma))) = N(A_{e_1}(A_{e_2}(\sigma)))$.*

Proof. By Theorem 10.6 (in its per-event form) both events satisfy strong absorption, and the chain of equalities in the proof of Theorem 10.10(c) applies in both directions. $\square$

In a registry satisfying CC2, then, CC1 is exactly commutation up to normalization, and footprints play no role in that equivalence; their role is to make (H4), or CC2 itself, a local check. Every part is mechanized: (a) `calc_footprint_cc1_iff`, (b) `calc_footprint_cc1_valid_iff` and `calc_footprint_cc1_valid`, (c) `calc_footprint_cc1` and `calc_fp_absorb_iff_sa`, Corollary 10.11 `calc_cc1_iff_commute_under_cc2`; with non-vacuity instances `calc_clamp_instance` (two clamped counters with disjoint footprints, CC1 at every state including invalid ones) and `calc_order_cc1`. Form (b) is the one the reference implementation's compositional check uses, since it checks CC1 on valid states. The same reasoning covers a state space split into components: on $\Sigma_1 \times \Sigma_2$ with $N = N_1 \times N_2$ and events acting on one component each, CC1 holds at every valid state, and at any state exactly when each event absorbs its own component's normalizer (`calc_components_cc1_valid`, `calc_components_cc1_iff`).

Remark 10.12 (Each Hypothesis Is Needed). Version 1 of this paper stated Theorem 10.10 with (H1) and (H2) alone as giving CC1 at every state. That is false. With no invariants ($k = 0$) every footprint is empty, repair is the identity, (H1) and (H2) hold vacuously, and the events $x := 1$ and $x := 2$ violate CC1 (`base_thm_footprint_cc1_refuted`). Adding raw commutation does not help: on states $\{0, 1, 2, 3\}$ with $\psi_1 = \psi_2 = (\sigma \geq 2)$, repairs $r_1 = r_2$ sending $0, 1 \mapsto 2$ and fixing 2, 3, and events $a = (0 \mapsto 0, 1 \mapsto 0, 2 \mapsto 2, 3 \mapsto 3)$ (footprint empty, repair-local) and $b = (0 \mapsto 0, 1 \mapsto 0, 2 \mapsto 3, 3 \mapsto 0)$ (footprint $\{1, 2\}$), the raw events commute but at state 0 the two sides of CC1 are 3 and 2 (`base_thm_footprint_cc1_raw_refuted`). The failure is that b does not absorb the repairs of its own footprint ($N(b(R_{\{1,2\}}(0))) = 3 \neq 2 = N(b(0))$), which is (H4); b also violates CC2 at 0. Dropping any one hypothesis of the corrected forms breaks them, with every other hypothesis holding: the valid-state form without (H3) (`calc_valid_needs_commute`), without (H1) (`calc_valid_needs_disjoint`), or without (H2) (`calc_valid_needs_repair_local`); the every-state form without (H3) (`calc_all_needs_commute`), without (H4) (`calc_all_needs_absorb`, the example above), or without (H2) (`calc_all_needs_repair_local`).

Remark 10.13 (Practical import). A practitioner does not prove CC1 by global state enumeration. Instead: (i) assign each event type an invariant footprint; (ii) verify that each event commutes with repairs outside its footprint (repair locality, Lemma 10.8); (iii) verify, per event, that it absorbs the repairs of its own footprint, (H4), equivalently strong absorption, which CC2 already implies; and (iv) verify, per independent pair, commutation up to normalization,

(H3). All four are local obligations, and (iv) is often raw commutation. At valid states, (ii) and disjoint footprints reduce CC1 to (iv) alone. The order fulfillment example (Section 4) does not have disjoint footprints: approval can falsify the single invariant and credit can restore it, so every footprint of either event contains it (`base_rem_practical_import_refuted`). It is certified by part (c) instead: the held-state repair commutes with credit application (`calc_order_credit_repair_commute`), both events absorb their footprint repairs, and the events commute up to normalization, so CC1 holds at every state, and CC2 holds as well (`calc_order_cc1`).

10.6 Product Composition

Real systems comprise multiple semantic modules. The following theorem shows CC verifications compose.

Definition 10.14 (Product Registry). Let $R_1 = (\Sigma_1, I_1, V_1, \rho_1)$ and $R_2 = (\Sigma_2, I_2, V_2, \rho_2)$ with disjoint invariant sets. Define $R_1 \otimes R_2$ by:

$$\Sigma = \Sigma_1 \times \Sigma_2, \quad V(\sigma_1, \sigma_2) = V_1(\sigma_1) \wedge V_2(\sigma_2), \quad \rho(\sigma_1, \sigma_2) = (\rho_1(\sigma_1), \rho_2(\sigma_2)).$$

Events are pairs $e = (e^{(1)}, e^{(2)})$ with application componentwise.

Theorem 10.15 (Product Lifting). *If each R_j satisfies WFC and CC, then $R_1 \otimes R_2$ satisfies WFC and CC; more precisely, CC1 holds for a product pair at (σ_1, σ_2) if and only if it holds for the first components at σ_1 and for the second components at σ_2 . If each normalizer is decomposable, the product normalizer is decomposable with footprints composed by union across components, and repair locality and footprint disjointness lift.*

Proof. **WFC:** Use measure $\Phi(\sigma_1, \sigma_2) = \Phi_1(\sigma_1) + \Phi_2(\sigma_2)$. Each compensation step strictly decreases at least one component when invalid.

CC2: The product normalizer satisfies $N(\sigma_1, \sigma_2) = (N_1(\sigma_1), N_2(\sigma_2))$ and application is componentwise, so CC2 holds iff it holds in each component.

CC1: Both sides of the CC1 equation decompose into paired equalities in R_1 and R_2 by componentwise application and normalization.

Decomposability: Per-invariant repairs in each component act on disjoint state components and thus commute across components trivially. The normalizer identity $N = N_1 \times N_2$ uses that each ρ_j fixes valid states (Definition 3.1). Mechanized as `base_thm_product` and `base_thm_product_decomposable`, with the instance `calc_product_instance`. $\square$

Remark 10.16. Product composition enables modular system design: verify CC for each semantic module independently, then compose without re-verification. An order processing system with separate inventory, payment, and shipping registries can verify CC per-module and lift to the combined system. This handles the *independent* case (disjoint invariants, no cross-module constraints). The federated convergence result (Section 8) generalizes product composition to the *dependent* case: registries connected by morphisms encoding cross-organizational semantic constraints.

10.7 Partial Decomposability

Full decomposability is a strong sufficient condition. When it does not hold globally, the calculus still provides value:

1. The footprint theorem (Theorem 10.10) is a statement about one event pair: it applies to any pair satisfying its hypotheses, regardless of whether *all* pairs do, and Corollary 10.11 needs no decomposability at all.
2. When footprints overlap for some event pair, the calculus identifies precisely which pairs require further analysis. A practitioner can then check the corrected canonical-state pattern for the overlapping invariants (Section 9.3, pattern 1) or add coordination only for the interacting event types.
3. For event types that share footprints but whose normalized outcomes agree by domain-specific reasoning (e.g., both repairs clamp to the same bound), CC1 can be verified by case analysis restricted to the overlapping states. The footprint calculus reduces the case space.

11 Conclusion

We have identified *normalization confluence* as a third structural regime for coordination-free convergence in distributed systems, alongside operation commutativity (CRDTs) and invariant confluence (I-confluence). The regime is characterized by two conditions: well-founded compensation (WFC), which ensures termination, and compensation commutativity (CC), which ensures confluence. Under both, the governance rewrite system has unique normal forms by Newman’s Lemma, and all processors consuming the same events converge to the same valid state. WFC and CC are sufficient for convergence; a uniform bound on compensation depth (UBC) is orthogonal to convergence and governs uniform complexity bounds. Without UBC, compensation depth is unbounded; without CC, different orders can reach different valid states. CC as stated over all states is sufficient but not necessary; its restriction to the states reachable from the start is exactly necessary and sufficient when every buffered event may fire and repair is canonical.

The result extends across organizational boundaries, in two layers. Repair confluence composes freely: on any acyclic network whose morphisms preserve validity (M1), with a source-determined (R1), validity-preserving (R2) *resolution operator* at each multi-source target, one round of federated normalization reaches the unique repair normal form from any state. The core mechanism is the authority argument: the sources’ states determine every target’s shared component. Event confluence does not compose for free. Version 1 of this paper claimed that federated convergence requires only validity preservation of the morphisms; that is false, because a target event can read a shared value that morphism repair overwrites. When events are applied to normalized states, two local checks per edge suffice: cross-registry compensation commutativity (C1) and repaired compensation commutativity (C2). The same two equations at the witnesses a run can reach are exactly necessary and sufficient, and dropping either one admits counterexamples that satisfy every component condition. When the shared domains are ordered lattices and repair is monotone, even *cyclic* networks have a unique repair normal form, the least fixed point (federally valid when the network is valid at bottom), which reveals acyclicity as a requirement of non-monotone repair only and exhibits order-independent repair as chaotic iteration. Event interleavings on a monotone cycle converge exactly when independent events commute after re-normalization, which per-target C1 and C2 over the repair images imply. Federated convergence is moreover *compositional*: a convex sub-federation certified by C1 and C2 on its own events collapses to a single effective registry with the same normal form and the same runs, so large networks verify modularly and hierarchically. The federated results, their counterexamples and their exact converses are mechanized in Coq/Rocq, axiom-free.

To make CC practically verifiable, we developed a verification calculus that reduces the global CC obligation to local checks. Invariant footprints identify which invariants each event can affect; decomposable repair ensures per-invariant fixes commute; CC2 is equivalent to strong

absorption; CC1 at every state follows from repair locality, per-event footprint absorption and commutation up to normalization, and at valid states disjoint footprints and repair locality reduce it to commutation up to normalization alone; and product composition enables modular verification across semantic modules. Each hypothesis is shown to be needed. Federated convergence generalizes product composition from independent registries to registries connected by directed semantic constraints, with C1 and C2 as the per-edge obligations that the connection adds.

Compensation commutativity is strictly weaker than the operation commutativity required by CRDTs: operations need not commute, only their compensated results. The gap between these two conditions is precisely the design space that normalization confluence occupies—systems where operations are non-commutative, invariant-violating, but convergent under repair.

The three regimes partition the coordination-free design space by where convergence is guaranteed: in the operations, in the invariants, or in the normalization rewrite system. This last regime formalizes a pattern that practitioners use implicitly—post-application repair as a convergence mechanism—and provides the structural conditions under which it is sound, a calculus for verifying those conditions, and a federated extension for applying them across organizational boundaries.

Acknowledgments

I thank my wife, Yuval, for her patience and support throughout this research.

References

- [1] Dayna Blackwell, “Normalization Confluence for Registry-Governed Stream Processing,” Zenodo, 2026. DOI: 10.5281/zenodo.18671870.
- [2] M. H. A. Newman, “On theories with a combinatorial definition of ‘equivalence’,” *Annals of Mathematics*, vol. 43, no. 2, pp. 223–243, 1942.
- [3] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “A comprehensive study of convergent and commutative replicated data types,” *INRIA Research Report RR-7506*, 2011.
- [4] J. M. Hellerstein, “The declarative imperative: experiences and conjectures in distributed logic,” *SIGMOD Record*, vol. 39, no. 1, pp. 5–19, 2010.
- [5] T. J. Ameloot, F. Neven, and J. Van den Bussche, “Relational transducers for declarative networking,” *Journal of the ACM*, vol. 60, no. 2, pp. 1–38, 2013.
- [6] P. Bailis, A. Fekete, M. J. Franklin, A. Ghodsi, J. M. Hellerstein, and I. Stoica, “Coordination avoidance in database systems,” *Proc. VLDB Endowment*, vol. 8, no. 3, pp. 185–196, 2014.
- [7] F. Baader and T. Nipkow, *Term Rewriting and All That*, Cambridge University Press, 1998.
- [8] W. Vogels, “Eventually consistent,” *Communications of the ACM*, vol. 52, no. 1, pp. 40–44, 2009.
- [9] H. Garcia-Molina and K. Salem, “Sagas,” *Proc. ACM SIGMOD International Conference on Management of Data*, pp. 249–259, 1987.

- [10] E. A. Lee and D. G. Messerschmitt, “Synchronous data flow,” *Proceedings of the IEEE*, vol. 75, no. 9, pp. 1235–1245, 1987.
- [11] G. Kahn, “The semantics of a simple language for parallel programming,” *Information Processing*, pp. 471–475, 1974.
- [12] T. Akidau, R. Bradshaw, C. Chambers, et al., “The Dataflow Model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing,” *Proc. VLDB Endowment*, vol. 8, no. 12, pp. 1792–1803, 2015.
- [13] L. Lamport, “Time, clocks, and the ordering of events in a distributed system,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.
- [14] A. Tarski, “A lattice-theoretical fixpoint theorem and its applications,” *Pacific Journal of Mathematics*, vol. 5, no. 2, pp. 285–309, 1955.
- [15] P. Cousot and R. Cousot, “Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints,” *Proc. 4th ACM SIGACT-SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pp. 238–252, 1977.
- [16] V. Balegas, S. Duarte, C. Ferreira, R. Rodrigues, N. Preguiça, M. Najafzadeh, and M. Shapiro, “Putting consistency back into eventual consistency,” *Proc. 10th European Conf. on Computer Systems (EuroSys)*, 2015.
- [17] V. Balegas, S. Duarte, C. Ferreira, R. Rodrigues, and N. Preguiça, “IPA: Invariant-preserving applications for weakly consistent replicated databases,” *Proc. VLDB Endowment*, vol. 12, no. 4, 2018.
- [18] J. Meseguer, “Conditional rewriting logic as a unified model of concurrency,” *Theoretical Computer Science*, vol. 96, no. 1, pp. 73–155, 1992.
- [19] F. Durán and J. Meseguer, “On the Church-Rosser and coherence properties of conditional order-sorted rewrite theories,” *Journal of Logic and Algebraic Programming*, vol. 81, no. 7–8, pp. 816–850, 2012.
- [20] E. W. Dijkstra, “Self-stabilizing systems in spite of distributed control,” *Communications of the ACM*, vol. 17, no. 11, pp. 643–644, 1974.
- [21] V. B. F. Gomes, M. Kleppmann, D. P. Mulligan, and A. R. Beresford, “Verifying strong eventual consistency in distributed systems,” *Proc. ACM on Programming Languages*, vol. 1, no. OOPSLA, art. 109, 2017.

A Errata and corrections

This appendix lists every statement of version 1 that version 2 refutes, corrects or makes precise. For each: the old statement (short), the counterexample with its Coq name, and the corrected result with its Coq names. All Coq names are theorems in the `Print Assumptions` gate of `coq/verify.sh`, closed under the global context. The body of the paper states only the corrected results.

A.1 Federated sections

The federated rewrite system (Definitions 8.5 and 8.6).

Old: $\mathcal{G}_{\text{Fed}}$ had apply steps “as in the single-registry case”, enabled at any state. *Change:* version 2 adopts the guarded system (apply steps only at federally valid states), the semantics of a federated processor that applies each event to a normalized state, under which C1 and C2 are exact. The unguarded system is treated in Remark 8.14: there C1 and C2 do not suffice (`fed_grs_c1_c2_insufficient`), and XU does (`fed_thm_fed_convergence_corrected`).

Lemma 8.9 (Authority Determination).

Old (c): the local component of a non-source “is unconstrained by morphisms and converges via component CC”. *Counterexample:* a two-registry tree meeting every hypothesis in which two event orders leave the target’s local counter at 0 and at 1 (`audit_counterexample`; against the exact hypotheses `fed_lem_authority_c_refuted`). *Corrected:* (c) states only that repair never modifies local components (`fed_lem_authority_c_local`); the local normal form is determined under C1 and C2 by Theorem 8.12 (under XU in the unguarded system, `fed_lem_authority_c_corrected`). (a) is stated precisely (a source’s component of any normal form is the unique normal form of its own single-registry system on its own events): `fed_lem_authority_a`; (b): `fed_lem_authority_b`.

Lemmas 8.10 and 8.23 (Single-Round Termination).

Old: one application of ρ_{Fed} yields a federally valid state. *Change (strengthened):* from *every* state, with the Phase 1 bound $\Phi_j(\sigma_j)$: `fed_lem_fed_termination`, `fed_lem_resolved_termination`, `fed_phase1_bound`. M1 is needed: `m1_single_round_fails`.

Theorem 8.11 (Federated CC).

Old: a tree-shaped network whose components satisfy WFC and CC satisfies federated CC. *Counterexamples:* `audit_counterexample` (C1 fails) and `c2_counterexample` (C1 holds, C2 fails); against the exact hypotheses `fed_thm_fed_cc_refuted`. *Corrected:* under (H), C1 and C2, the governed step satisfies CC1 at every federally valid state (`fed_thm_fed_cc_corrected_machine`, `fed_events_commute`); under XU, CC1 and CC2 hold at every state (`fed_thm_fed_cc_corrected`).

Theorem 8.12 (Federated Normalization Confluence).

Old: tree, M1, component WFC and CC imply that processors consuming the same events converge. *Counterexamples:* `fed_thm_fed_convergence_refuted` (C1 fails) and `fed_c2_paper_counterexample` (C2 fails). *Corrected:* with C1 and C2, the guarded system has unique, federally valid normal forms (`fed_thm_fed_convergence_guarded`); exactly, iff C1 and C2 hold at the reachable witnesses (`fed_thm_fed_convergence_exact`, `fed_guarded_exact`, `fed_exact`, `fed_exact_full`, `acyclic_gc_iff`). Non-vacuity: `fed_supply_paper_instance`, `ms_converges`.

Corollary 8.15 (Constructive Normal Form).

Old: compute sources, propagate shared components, “let each non-source converge its local component via component CC”. *Counterexample:* even under C1 and C2 the recipe “run the target’s events as a single registry, then overwrite its shared component” gives a state that no event order reaches (`fed_cor_fed_nf_refuted`). *Corrected:* the normal form is the unique solution of the equations of Corollary 8.15, with repair interleaved after every target event (`fed_cor_fed_nf_corrected`, `fed_nf_constructive`); the old recipe holds under XU (`fed_nf_recipe_xu`).

Theorem 8.24 and Corollary 8.25.

Old: the resolved analogues of Theorem 8.12 and Corollary 8.15, under M1, R1, R2 and component WFC and CC. *Counterexample:* the tree counterexamples are resolved networks

(`fed_thm_resolved_convergence_refuted`). *Corrected:* `fed_thm_resolved_convergence_guarded`, `fed_thm_resolved_convergence_exact`, `fed_cor_resolved_nf_corrected`; unguarded under XU, `fed_thm_resolved_convergence_corrected`.

Remark “Necessity of the resolution conditions”.

Old: “(R1) is necessary for well-definedness” (of an unspecified object). *Made precise:* without (R1) the repair normal form is not unique: two federally valid normal forms with different shared components from one state (`r1_necessary`). The (R2) claim is mechanized (`r2_necessary`).

Theorem 8.29 (Monotone Convergence Despite Cycles).

Three parts of the old statement fail. (i) *Old:* “validity is preserved along the iteration by M1/R2, so $\bar{s}^*$ is federally valid”. *Counterexample:* a 2-cycle of identity morphisms whose least fixed point is invalid (`fed_thm_monotone_cycles_lfp_invalid`). *Corrected:* add validity at bottom (`net_iter_valid`, `net_sweep_valid`, `net_lfp_valid`, `net_Ncyc_correct`, `net_Ncyc_idem`). (ii) *Old:* $\bar{s}^* = \bigsqcup_k \Phi^k(\perp)$ on any complete lattice. *Counterexample:* a monotone map on $0 < \dots < \omega < \omega+1$ whose Kleene supremum is not fixed (`fed_thm_monotone_cycles_kleene_formula_fails`). *Corrected:* the formula needs Φ to preserve the supremum of the Kleene chain (`kleene_sup_lfp`, `kleene_sup_valid`); finite reachability needs ACC (`kleene_acc_lfp`, `chaotic_acc_reaches_lfp`). (iii) *Old:* “consequently all processors consuming the same federated events converge”. *Counterexample:* a monotone 2-cycle meeting every hypothesis on which one allowed swap of two events diverges (`cyc_counterexample`, `fed_thm_monotone_cycles_events_refuted`). *Corrected:* convergence iff GC (`cyc_events_converge_iff`, `net_events_converge_iff`); per-target C1 and C2 over the repair images imply GC (`cyc_check_gc`, `cyc_check_gc_lfp`). The source entry of Φ in Definition 8.28 now keeps the source’s own shared value, as in the mechanized operator (`fed_def_lattice_shared_cyclic_hyps`).

Corollary 8.30.

Old: acyclicity and monotonicity are two independent routes to “federated confluence”. *Made precise:* to a unique repair normal form; event confluence needs C1 and C2 or GC in addition (counterexamples as for Theorems 8.12 and 8.29; `negation_not_monotone`).

Remark 8.34 (Infinite lattices and widening).

Old: “the convergence guarantee is unaffected” without ACC, and widening yields a sound post-fixed point. *Made precise:* without ACC no iterate need reach the least fixed point even for continuous Φ (`kleene_sup_instance`); a widened post-fixed point bounds the least fixed point (`widening_sound`) but need not be a fixed point, so it is not the federated normal form (`widening_not_normal_form`).

Theorem 8.37 (Compositional Collapse) and Corollary 8.38.

Old (a): under M1/R2 and component WFC and CC, internal convergence makes $R_{\mathcal{J}}$ satisfy CC. *Counterexample:* the federation of Remark 8.8 as $\mathcal{J}$ (`collapse_a_refuted`). *Corrected:* WFC as stated (`collapse_a_wfc`); CC under C1($\mathcal{J}$), C2($\mathcal{J}$) in the guarded form, exact at reachable witnesses (`collapse_a_guarded`, `collapse_a_guarded_exact`). *Old proof step (c):* a topological order visits convex $\mathcal{J}$ as a contiguous block. *Counterexample:* not every order does (`collapse_every_order_refuted`); *corrected:* some order does (`collapse_block_order`). *Old (b):* boundary M1 iff. *Counterexamples* under Definition 8.36’s validity (`collapse_b_import_refuted`, `collapse_b_export_refuted`); *corrected:* exact under the product reading (`collapse_b_import_iff`, `collapse_b_export_iff`), and the restoration form (`collapse_b_import_corrected`, `collapse_b_export_corrected`). (c) holds with C1 and C2 in place of component CC (`collapse_nf_agree`, `collapse_c_guarded_iff`, `collapse_c_guarded_exact`); Corollary 8.38: `collapse_modular_converges`, `collapse_hierarchical`. The cyclic monotone case is on paper only.

Remark “Convexity and the monotone regime”.

Old: if the sub-federation’s repair is monotone then so is $\rho_{\text{Fed}}^{\mathcal{J}}$. *Counterexample:* `fed_rem_convexity_refuted` (Phase 1 is not monotone). *Corrected:* monotone Phase 1 and Φ monotone in the locals as well give a monotone $\rho_{\text{Fed}}^{\mathcal{J}}$ under ACC (`fed_rem_convexity_corrected`, `lfp_mono_param`).

Abstract, contributions 7 to 10, Remark 9.1, Section 9.2 and Table 1, the supply-chain application, and the conclusion.

Old: “federated convergence requires only validity preservation of the morphisms”; “multi-registry systems inherit convergence from their components”; monotone repair gives convergence on any topology. *Corrected:* repair confluence composes under M1/R1/R2 on acyclic networks (and under monotonicity with validity at bottom on cycles); event confluence needs C1 and C2 (acyclic) or GC (monotone cycles). The counterexamples are those listed above.

Examples.

Proposition 8.18, Proposition 8.19 and the manufacturer and supplier federation are unchanged and now mechanized (`prop_cycle_necessary`, `prop_m1_necessary`, `ms_instance`, `ms_converges`).

A.2 Restated single-registry sections

The following items concern the sections restated from [1] (Sections 3 to 7, 9 and 10), which are corrected identically in its version 2.

E1. Theorem 10.10 (Footprint Commutativity). *Old:* under WFC and decomposable repair, disjoint footprints (H1) and repair locality (H2) give CC1 for an independent pair. *Counterexamples:* with $k = 0$ the events $x := 1$ and $x := 2$ satisfy (H1) and (H2) vacuously and violate CC1 (`base_thm_footprint_cc1_refuted`); the four-state example of Remark 10.12 satisfies (H1), (H2) and raw commutation and violates CC1 at state 0 (`base_thm_footprint_cc1_raw_refuted`). The old proof’s final step moved a repair block past an event whose footprint it intersects, which repair locality does not allow. *Corrected:* Theorem 10.10: (a) under (H1), (H2), CC1 at σ iff the events commute up to N after the other event’s footprint repairs (`calc_footprint_cc1_iff`); (b) at valid states, CC1 iff commutation up to N (`calc_footprint_cc1_valid_iff`, `calc_footprint_cc1_valid`); (c) at every state, repair locality, commutation up to N and per-event footprint absorption give CC1, without (H1) (`calc_footprint_cc1`, `calc_fp_absorb_iff_sa`); and Corollary 10.11: under CC2, CC1 iff commutation up to N (`calc_cc1_iff_commute_under_cc2`). Each hypothesis is needed (`calc_valid_needs_commute`, `calc_valid_needs_disjoint`, `calc_valid_needs_repair_local`, `calc_all_needs_commute`, `calc_all_needs_absorb`, `calc_all_needs_repair_local`).

E2. Remark “Practical import”. *Old:* the order-fulfillment events have disjoint footprints, so the footprint workflow certifies the example. *Counterexample:* the single invariant lies in every footprint of both events (`base_rem_practical_import_refuted`). *Corrected:* Remark 10.13: the workflow has a fourth step (per-event footprint absorption), and the example is certified by Theorem 10.10(c) (`calc_order_credit_repair_commute`, `calc_order_cc1`).

E3. Section 9.3, pattern 1 (compensation to a canonical state). *Old:* if $\rho_R^*(\sigma) = \sigma_{\perp}$ for every invalid σ , CC1 and CC2 hold trivially. *Counterexamples:* $x := 1$ and $x := 2$ on valid states violate CC1 (`base_pattern1_cc1_refuted`); an event that sends an invalid state and the reset state to different places violates CC2 (`base_pattern1_cc2_refuted`).

Corrected: CC2 for e iff $N(A_e(\sigma)) = N(A_e(\sigma_\perp))$ for every invalid σ (`calc_canonical_cc2_iff`); given that for both events, CC1 iff commutation up to N (`calc_canonical_pattern`, `instance calc_canonical_instance`).

E4. Section 9.3, pattern 2 (lattice-structured compensation). *Old:* compensation that “computes the join with the nearest valid state” satisfies CC by commutativity of the join. This was imprecise (“nearest valid state” is undefined). *Corrected:* stated precisely (events join fixed elements, N a closure operator, $\sigma \sqsubseteq \rho_R(\sigma) \sqsubseteq N(\sigma)$) with a short paper proof; not mechanized. The federated remark in the same item no longer claims event-order convergence on monotone cycles, which needs a further condition (Theorem 8.29(c)).

E5. Section 4, “Why Compensation Design Matters”. *Old:* under the naive revert $\rho(\text{approved}, b) = (\text{pending}, b)$, CC1 fails at $(\text{pending}, 0)$. *Counterexample:* at $(\text{pending}, 0)$ both orders give $(\text{pending}, 1)$ (`naive_paper_witness_fails`). *Corrected:* CC1 fails at $(\text{pending}, 1)$, with $(\text{pending}, 2) \neq (\text{approved}, 2)$ (`naive_cc1_fails`); the paper’s stream diverges (`naive_stream_diverges`); the naive revert also violates CC2 at $(\text{approved}, 1)$ (`naive_cc2_fails`). The closing paragraph no longer cites “state-determined compensation” as a sufficient condition (canonical compensation is a special case and is not sufficient, E3); it names the absorption and commutation facts that Theorem 10.10 uses. The exhaustive check covers all 12 states (`of_cc1`, `of_cc2`, `of_processors`).

E6. Remark “What Fails” (after Theorem 6.1). *Old:* R_∞ satisfies CC because “there is only one event per set, so order independence is vacuous”. The conclusion is true, the reason is not (CC1 is a property of the registry over all events, and $\text{apply}(e_n, -)$ was defined only at 0). *Corrected:* ρ^* is constantly 0, so CC1 and CC2 hold for every total extension of $\text{apply}(e_n, -)$ (`ri_cc_any_extension`, `ri_what_fails`); stronger forms of the UBC failure: `ri_depth_exact`, `ri_no_ubic`. Theorem 6.1 itself is unchanged (`thm_necessity`).

E7. Theorem 5.5(c) (Agreement). *Old:* if $E_{P_1}(t) = E_{P_2}(t)$ then $\Psi_{P_1}(t) = \Psi_{P_2}(t)$. *Counterexample:* one processor has applied the single received event and the other has not yet (`base_thm_convergence_c_counterexample`). *Corrected:* processors that have *finished processing* equal received sets agree, at the same or different times (`base_thm_convergence_c`, `stream_agreement`); parts (a) and (b) are stated for finished processors (`base_thm_convergence`, `stream_convergence`). Definition 3.19 now defines “finished” and “fair”.

E8. Closing sentence of Theorem 5.5 and Remark 5.9. *Old:* with eventual delivery, disagreements are transient: any event causing processors to differ is eventually received by both, restoring agreement. *Counterexample:* on an infinite stream, a processor one event behind another is finished at every time and disagrees at every time (`base_thm_convergence_transient_counterexample`). *Corrected:* the sentence is removed; Remark 5.6 and Remark 5.9 state what holds: (c) as corrected, and fixed-point agreement for finite streams.

E9. Corollary 5.8. *Old:* for a finite stream the processors eventually agree forever. The proof assumed both processors had finished processing at T_N . *Corrected:* the same conclusion for fair processors (`base_cor_quiescent`, `base_cor_quiescent_nat`).

E10. “CC is necessary” (abstract, contributions, Section 6, the remark after Proposition 6.3, conclusion). *Old:* CC is necessary for agreement, and each condition is independently necessary for its guarantee. Proposition 6.3 is existential and remains true (`prop_cc_necessary`), but CC as stated is not necessary in the paper’s causal setting. *Counterexamples:* under causal enabledness CC1 can fail at a reachable configuration while normal forms are unique (`masked_cc1`); with non-canonical ρ^* the same

(`rho_star_qualifier`). *Corrected:* Theorem 6.5: under free delivery and canonical repair, unique normal forms from σ_0 iff CC1 and CC2 on the states reachable from σ_0 (`cc_exact_from`, `wfc_cc_exact_from`); under any enabledness, confluence iff joinability at reachable configurations (`jc_exact`); Remark 6.6.

E11. Theorem 10.6. *Old:* strong absorption implies CC2. *Sharpened:* under WFC the two are equivalent (`strong_absorption_iff_cc2`; forward direction `base_thm_strong_absorption`; per event `calc_cc2_strong_absorption`).

E12. Three-regime hierarchy (Section 1, Section 9.1, Related Work). *Old:* the regimes form a containment hierarchy of progressively weaker requirements, and normalization confluence subsumes invariant confluence. This was imprecise: an invariant-confluent system needs no compensation but its operations need not commute, so it can violate CC1. *Corrected:* normalization confluence contains the invariant-confluent systems that converge under reordering; the op-based CRDT side is exact (`compensation_free_exact`, `cmrdt_governed_SEC`, `witness_converges`, `witness_not_cmrdt`).

E13. Sharpened or newly mechanized, no statement refuted. Definition 3.15: ρ_R^* is constructed from WFC, needs at most $\Phi(\sigma)$ steps, and is independent of the measure (`base_def_rhostar`, `base_def_rhostar_least_unique`, `rho_star_canonical`). Lemma 5.1: the bound is attained (`base_lem_termination_bound`, `lv_termination_bound_tight`). The infinite-domains corollary of [1], not restated here, is mechanized at the rewrite-system and stream levels (`governance_wf_confluent`, `base_cor_infinite`, `zw_confluent`). Theorem 10.15: CC1 lifts as an equivalence with the componentwise conditions (`base_thm_product`, `base_thm_product_decomposable`). The Applications paragraphs no longer assert CC for reset and per-constraint remediation; they point to the corrected checks. The remark “Machine-checked” (Remark 5.4) lists the Coq theorem names; version 1’s phrase “footprint disjointness implies order-independent commutation” referred to state-variable footprints (`disjoint_events_commute`), not to Theorem 10.10.